# Algorithmic Pricing with AI Agents: A Comparative Analysis of Deep Reinforcement Learning Approaches

Master's Thesis

Presented to the
Department of Economics at the
University of Bonn

In Partial Fulfillment of the Requirements for the Degree of
Master of Science (M.Sc.)

Supervisor:

Dr. Ivan Gufler

Submitted in June 2026 by

Aditya Goswami

Matriculation Number: 50203608

# Contents

# 1 Introduction

Prices in securities markets are increasingly set by algorithms. For instance, the SEC report (SEC 2020, p. 5) notes that the use of algorithms in trading is pervasive in today's markets, with advances in technology enabling market participants to more efficiently provide and access liquidity, implement new trading services, and more effectively manage risk.

This evolution raises concerns about the impact of AI-powered algorithms on market efficiency, liquidity and stability. To assess whether such concerns are well-founded, it is essential to develop a deeper understanding of algorithmic behaviour in financial markets, in order to predict and explain their influence on market outcomes (Goldstein et al. 2021; Colliard et al. 2026).

In my experiments, Artificial Intelligence (AI)-powered algorithms play a market making game identical to the one proposed by Colliard et al. (2026) which is similar to the one proposed by Glosten and Milgrom (1985). These algorithms have two key features of reinforcement learning: (i) they update the estimated value of an action based on the observed outcomes when that action is taken, and (ii) they engage in limited experimentation. The main finding from my experiments is that while a relatively simple algorithm fails to learn competitive pricing strategies due to noisy feedback regarding the profitability of undercutting a competitor, a relatively more complex reinforcement learning algorithm performs comparatively better by closely approximating the benchmark values and rightly undercutting their competitor to achieve equilibrium under identical market conditions.

Some background of the market making game I will be looking at is a version of the $n$-Armed Bandit Problem popular in reinforcement learning literature (see Sutton, Barto, et al. 1998). The $n$-armed bandit problem captures a fundamental challenge in decision making for reinforcement learning algorithms: how to balance exploration or experimentation vs exploitation. This exploration-exploitation trade-off is central to learning in uncertain environments. The problem can be described using a simple analogy of a slot machine, or "one-armed bandit", except that instead of one lever it has $n$ levers. Each action is like pulling the lever of one slot machine and the reward is the payoff for hitting the jackpot, however, you do not know in advance which "levers" or options will be the most rewarding. Through repeated trial-and-error action selections you can maximise your winnings by concentrating your actions on the best levers.

Formally, you are faced repeatedly with a choice among $n$ different actions. After each action you receive a stochastic reward from a stationary probability distribution that depends on the action taken. The objective is to maximise the expected total reward over some time period, for example, over 1,000 action selections, or *iterations*.

To explain the $n$-armed bandit problem in the context of market making, each action has an expected or mean reward given that the action is selected which can be called *value* of the action. It

is assumed that the value of any action is not known with certainty, however, there may be estimates. Then, at any time there is at least one action whose estimated value is the greatest. This can be called *greedy* action. If the greedy action is selected, then you are *exploiting* your current knowledge of the values of actions. Instead, if one of the nongreedy actions is selected then you are *exploring*, because this allows you to improve your estimate of the action values.

In the baseline version of the market making game (Colliard et al. 2026), two dealers receive quote requests from clients who wish to buy one share of a risky asset. Dealers are uninformed about the payoff of the asset and simultaneously respond with an offer. Each client buys if the best offer is less than her valuation for the asset, which is the sum of the payoff of the asset and a private valuation (the client's "liquidity shock"), and does not trade otherwise. Thus, holding the dealers' price constant, a client is more likely to buy the asset when its payoff is high rather than when its payoff is low. Dealers are therefore exposed to adverse selection. After each client arrival, the asset pays off, and the dealers receive their realized profit — that is, the difference between the selling price and the asset's payoff if the dealer sold the asset, and zero otherwise.

Importantly, two competing dealers have no prior knowledge of the primitives of the game — the distribution of the asset value, the dispersion of client's liquidity shocks, or the pricing strategy of their competitor. Each dealer sets prices using a reinforcement learning algorithm that learns from realised trading outcomes. I study two such algorithms — Q-learning (Watkins and Dayan 1992), a foundational reinforcement learning algorithm, and the Deep Deterministic Policy Gradient algorithm (Lillicrap et al. 2019) and examine how closely each tracks the competitive Nash equilibrium price predicted by the standard economic analysis of the game.

The key challenge for any reinforcement learning algorithm in this setting is that feedback about the value of competitive pricing is noisy. When a dealer quotes a competitive price, the realised profit may be low not because the price is unprofitable in expectation, but because the asset value happened to be high or the client chose not to trade. Q-learning, which updates values based on realised rather than expected profits is particularly susceptible to this noise. Competitive prices are prematurely discouraged before their true expected profitability can be accurately assessed. The DDPG algorithm, in contrast, which employs a neural function approximator and operates over a continuous action space, provides smoother gradient signals that are less sensitive to individual noisy realisations, potentially allowing it to more accurately assess the profitability of competitive pricing.

The choice of algorithms is motivated by two considerations. First, Q-learning serves as the natural baseline given its extensive use in the algorithmic market making literature (Calvano et al. 2020; Dou et al. 2025; Colliard et al. 2026). Second, Deep Deterministic Policy Gradient represents the most direct extension of Q-learning to continuous action spaces, it preserves the off-policy learning

of Q-learning while replacing the discrete Q-matrix with a neural function approximator that operates over continuous action spaces (Lillicrap et al. 2019). This makes the comparison clean and interpretable while the key difference is the discrete versus continuous action space.

In general, it is observed that Q-learning dealers settle on prices well above the Glosten-Milgrom benchmark, consistent with the findings of Colliard et al. (2026). The DDPG algorithm, by contrast, converges to a mean price close to the competitive Nash equilibrium across a range of market parameters. This difference in benchmark tracking emerging from two algorithms playing an identical game under identical conditions suggests that the architectural properties of the learning algorithm are a first-order determinant of the results.

Huck et al. (2004) find that markets with four or five firms are never collusive and typically settle at or above the competitive outcome. However, even at $N = 5$ dealers, Q-learning prices remain substantially above the competitive benchmark consistent with Colliard et al. (2026), suggesting that market structure alone is insufficient to overcome noisy feedback. The DDPG algorithm tracks the benchmark closely regardless of the number of competing dealers, indicating that algorithm sophistication may be a more effective route to competitive market making than increasing market participation alone.

Following Colliard et al. (2026), I examine how the dispersion of the clients' liquidity shock $\sigma$ affects pricing outcomes. Dou et al. (2025) find that aggressive trading strategies are more likely to be prematurely pruned. Intuitively, higher values of $\sigma$ amplify the likelihood of competitive prices being pruned earlier. Consistent with their findings, Q-learning prices become less competitive as $\sigma$ increases, the noise in the feedback received by the algorithm also increases, impairing its ability to assess the profitability of competitive pricing. The DDPG algorithm, by contrast, tracks the Glosten-Milgrom benchmark across all values of $\sigma$ examined, suggesting its neural function approximation is more robust to noisy feedback than the tabular Q-value updates of Q-learning.

This thesis makes two contributions. First, it provides the direct comparison of Q-learning and DDPG in a Glosten-Milgrom market making setting (Glosten and Milgrom 1985; Colliard et al. 2026), demonstrating that algorithm sophistication has first-order consequences for how closely algorithmic dealers track the competitive benchmark. Second, it shows that this result is robust to changes in market structure and the degree of adverse selection, suggesting that the architectural properties of the learning algorithm are a more important determinant of market quality than the number of competing dealers or the level of market uncertainty.

The rest of the thesis is structured as follows: Section 2 reviews relevant literature. Section 3 describes the market making game I will be studying in this paper, taken from Colliard et al. (2026). Section 4 outlines the problem the algorithms have to solve and details the ways different algorithms approach the problem. Section 5 explains the design of the algorithms that tackle the market making

game. In Section 6 I justify the choice of hyperparameters for both algorithms. Section 7 presents the findings from theory described in previous sections. Section 8 provides more analysis on the collusive outcome of AI agents. Section 9 discusses the findings and the scope for further research as well as limitations of the methods applied in this thesis followed by concluding remarks in section 10. Finally, the Appendix provides additional figures for both algorithms and the code used to produce the results reported in this thesis to facilitate reproducibility.

## 2 Review of Literature

This thesis builds on the work by Colliard et al. (2026) who are the first to study how Q-learning algorithms for market making behave in an environment with uncertainty about demand and costs, due to asset volatility and adverse selection. One branch of the literature on algorithmic pricing shows that Q-learning algorithms can learn to play collusive strategies (see, e.g., Calvano et al. 2020; Dou et al. 2025). There are various interpretations of the collusive behaviour observed in this model. The findings of Colliard et al. (2026) are consistent with what Abada et al. (2024) term "collusion by mistake" (see also Asker et al. 2024). Dou et al. (2025) study how informed traders using Q-learning algorithms behave in a Kyle (1985) environment. While their setting differs from the present thesis, the analyses are complementary — the collusion results documented here are interpreted through the concept of the "Artificial Stupidity" mechanism presented in their analysis.

This thesis also contributes to the broader literature on algorithmic trading using machine learning techniques. The theoretical literature explores how machine learning (e.g., Gu et al. (2020), Kelly and Xiu (2023)) and reinforcement learning (e.g., Li et al. (2019), Zhang et al. (2019)) can be used to implement profitable strategies in trading environments.

In contrast to this literature, the approach in this thesis is experimental. In the same way in which economists use experiments to understand human behaviour and how it can differ from standard economic analysis, the present thesis runs simulation experiments to understand algorithmic behaviour in a controlled environment as Colliard et al. (2026). The Nash equilibrium of the market making game serves as an important benchmark to identify outcomes that are not predicted by standard economic analysis. Specifically, the emergence of supra-competitive pricing in the absence of any explicit communication among dealers.

The present thesis extends this experimental framework by introducing the Deep Deterministic Policy Gradient algorithm of Lillicrap et al. (2019) as an alternative market making strategy. The objective is not solely to study which algorithm converges to the Nash equilibrium but rather to examine whether the architectural differences between Q-learning and DDPG are sufficient to overcome

collusive behaviour observed in Q-learning (Calvano et al. 2020; Dou et al. 2025; Colliard et al. 2026) and assess how both algorithms respond to changes in the economic environment as a comparative exercise.

# 3 The Market Making Game

This section describes the market making model developed by Colliard et al. (2026), where algorithmic market makers use reinforcement learning to set optimal prices in an uncertain environment, with asset volatility and adverse selection. Following the game, the "Glosten-Milgrom price" (Glosten and Milgrom 1985) obtained from the Nash equilibrium of the game is derived which serves as a useful benchmark for interpreting algorithmic market makers' behaviour.

## 3.1 The Market Making Game with Adverse Selection

Following the framework of Colliard et al. (2026) , one investor ("client") wants to buy one share of a risky asset. The asset payoff, $\tilde{v}$, has a binary distribution: $\tilde{v} \in \{v_H, v_L\}$, with $\Delta_v := v_H - v_L \geq 0$ and $\mu := Pr(\tilde{v} = v_H)$. The payoff is realized before trading starts and is only revealed after trading has taken place or not.

The client privately knows her own valuation of the asset, which is equal to $\tilde{v}^C = \tilde{v} + \tilde{L}$, where $\tilde{L}$ is referred to as the client´s "Liquidity Shock" which is normally distributed with mean zero and variance $\sigma^2$, and is independent from $\tilde{v}$. The c.d.f. of the client's liquidity shock is therefore denoted by $G(.)$. The distribution of $\tilde{v}^C$ is therefore a mixture of two normal distributions with means $v_L$ and $v_H$, respectively, as shown in figure 1.

After observing her valuation, the client requests quotes from $N$ dealers, who simultaneously respond by posting a price ($a_n$ for dealer $n$) at which they are willing to sell one unit of the asset. The vector of prices is denoted $\overline{a} = \{a_n\}_{1 \leq n \leq N}$, $a^{min} := \min_n \{a_n\}$ the best offer, and $N^{min}$ the number of dealers posting the best offer. The asset payoff is disclosed to dealers after the client´s decision (buy/no buy).

The client buys if and only if the best offer is less than her valuation of the asset ($a^{min} \leq \tilde{v}^C$). Let $V(a^{min}, \tilde{v}^C)$ be the client´s realized demand (or trading volume). It is 1 if the client buys the asset and 0 otherwise. Dealer n´s realized trading volume is

$$I(a_n, \overline{a}, \tilde{v}^C) = V(a^{min}, \tilde{v}^C) Z(a_n, \overline{a}), \tag{1}$$

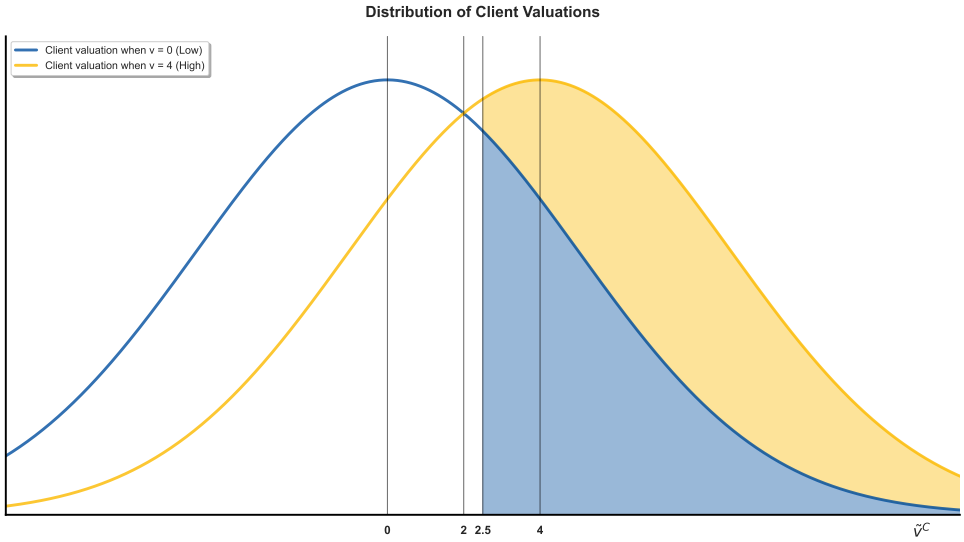


**Figure 1. Distribution of the client's valuation**, $\tilde{v}^C$. Parameter values are $v_H = 4$, $v_L = 0$, $\mu = \frac{1}{2}$, and $\sigma = 5$. This figure displays the distributions of clients' valuations for $v = v_H$ (yellow) and $v = v_L$ (blue). The former is shifted to the right relative to the latter, and the distributions partially overlap. If dealers' best offer is $a^{\min} = 2.5$, the likelihood that the client buys the asset is given by (i) the area (in blue) to the right of 2.5 and under the blue curve when $v = v_L = 0$, and (ii) the area (in blue and yellow) to the right of 2.5 and under the yellow curve when $v = v_H = 4$.

where $Z(a_{min}, \overline{a}) = \frac{1}{N^{min}}$ if $a_n = a^{min}$ (the client´s demand is split equally among the dealers posting the best offer) and $Z(a_n, \overline{a}) = 0$ otherwise. Hence, dealer n´s realized profit is

$$\Pi(a_n, \overline{a}, \tilde{v}^C, \tilde{v}) := I(a_n, \overline{a}, \tilde{v}^C_\tau)(a^{min} - \tilde{v}) \tag{2}$$

Importantly, in this game, dealers are exposed to adverse selection. Therefore, holding the best offer constant, the client is more likely to buy the asset when its payoff is high rather than when its payoff is low. To see this, let $D(a^{min}, v) := Pr(a^{min} \leq \tilde{v}^C | \tilde{v} = v)$ be the probability that the client buys the asset when its payoff is $v$. Therefore,

$$D(a^{min}, v) := Pr(a^{min} \leq \tilde{v}^C | \tilde{v} = v) = Pr(a^{min} \leq \tilde{v} + \tilde{L}) = 1 - G(a^{min} - v). \tag{3}$$

Thus, holding the best price constant, $D(a^{min}, v_H) > D(a^{min}, v_L)$.

## 3.2 The Market Making Game without Adverse selection

The market making game experiments are designed to understand how dealers' exposure to adverse selection affects their prices; it is therefore useful to have a benchmark market making game without adverse selection. Hence, Colliard et al. (2026) design a market making game identical to the one

described in Section 3.1, with the exception that the clients' valuation is $\tilde{v}^C = \tilde{w}^C + \tilde{L}$ where $\tilde{w}^C$ and $\tilde{v}^C$ are i.i.d.

In this case, there is no adverse selection since the client's decision to buy does not depend on $\tilde{v}$. Thus, the likelihood that the client buys the asset is the same whether the asset payoff is high or low. Nevertheless, the likelihood is the same as when there is adverse selection versus when there is not because the unconditional distribution of the client's valuation of the asset is exactly the same in both cases. The likelihood of a buy is $\mathbb{E}_\mu(V(a^{min}, \tilde{v}^C)) := \mu D(a^{min}, v_H) + (1-\mu)D(a^{min}, v_L)$, in either case since $\tilde{w}^C$ has the same distribution as $\tilde{v}^C$. This point is important since it allows for comparisions between experimental outcomes with and without adverse selection, holding all other parameters of the environment unchanged. Consider an alternative approach to eliminate adverse selection such as increasing the standard deviation $\sigma$ of liquidity shocks. As $\sigma$ grows large, the distributions of client valuations under $v = v_H$ and $v = v_L$ increasingly overlap (yellow and blue area in figure 1) and such an increase reduces adverse selection. However, an increase in $\sigma$ also changes the likelihood of a buy, since it alters the distribution of the clients' valuation for the asset. For example, consider a dealer quoting $a = 3$ under two values of $\sigma$: At $\sigma = 5$, the probability of a trade is approximately $\approx 0.427$. At $\sigma = 9$, this probability rises to approximately $\approx 0.489$. The two environments differ not only in adverse selection but also how frequently a trade is made. Any difference in algorithmic behavior across these two scenarios could reflect either the reduction in adverse selection or the increase in trading frequency, making it impossible to cleanly attribute the result to one cause.

The no adverse selection environment used avoids this problem by removing the client's trading decision from the asset value while holding the trading frequency fixed at exactly the same level as the adverse selection case. This allows for a clean comparison where adverse selection is the only thing that differs between the two conditions.

### 3.3 Glosten-Milgrom Benchmark

I benchmark the outcomes of the experiments against the "Glosten-Milgrom price" (Glosten and Milgrom 1985; Colliard et al. 2026) obtained in the Nash-equilibrium of the market making game. This approach allows us to uncover unique decisions of algorithms relying on experimentation-exploitation methods to set prices.

From (2), dealer $n$´s expected profit, $\overline{\Pi}(a_n, \overline{a}; \mu) := \mathbb{E}_\mu(\Pi(a_n, \overline{a}, \tilde{v}^C_\tau, \tilde{v}^C))$, is

$$\overline{\Pi}(a_n, \overline{a}; \mu) = Z(a_n, \overline{a})[\mu D(a^{min}, v_H)(a^{min} - v_H) + (1-\mu)D(a^{min}, v_L)(a^{min} - v_L)], \quad (4)$$

which can be written as,

$$\overline{\Pi}(a_n, \overline{a}; \mu) = \underbrace{Z(a_n, \overline{a})\mathbb{E}_\mu(V(a^{min}, \tilde{v}^C))}_{\text{Dealer's expected trading volume}} \left[ \overbrace{(a^{min} - \mathbb{E}_\mu(\tilde{v}))}^{\text{Quoted Spread}} - \underbrace{\Delta_v \frac{(1-\mu)\mu\Delta_D(a^{min})}{\mathbb{E}(V(a^{min}, \tilde{v}^C))}}_{\text{Adverse Selection Cost}} \right], \tag{5}$$

where $\mathbb{E}_\mu(\tilde{v}) = \mu v_H + (1-\mu)v_L$, and $\mathbb{E}_\mu(V(a^{min}, \tilde{v}^C)) := \mu D(a^{min}, v_H) + (1-\mu)D(a^{min}, v_L)$ are the expected asset value, and the likelihood to buy, respectively, and $\Delta_D(a^{min}) := D(a^{min}, v_H) - D(a^{min}, v_L)$. The term in brackets (5) is dealer *n*'s expected profit per share conditionally on a trade.

Let *a** be the lowest price such that if $a^{min} = a^*$ then dealers obtain zero expected profits ($\overline{\Pi}(a_n, \overline{a}; \mu) = 0$). From (5), it can be deduced,

$$a^* = \mathbb{E}_\mu(\tilde{v} \mid a^* \leq \tilde{v}^C) = \mathbb{E}_\mu(\tilde{v}) + \underbrace{\Delta_v \frac{(1-\mu)\mu\Delta_D(a^*)}{\mathbb{E}_\mu(V(a^*, \tilde{v}^C))}}_{\text{Adverse Selection Cost}} \tag{6}$$

The zero expected profit price is the expected payoff of the asset conditional on the client buying the asset, as given in Glosten and Milgrom (1985). This is why Colliard et al. (2026) call this the Glosten-Milgrom price. In this setting, using Bertrand logic suppose a contradiction, where Nash equilibrium is at $a > a^*$. Then, one dealer can undercut the price by some infinitesimal amount, she increases by $\frac{N-1}{N}$ her expected profit since she captures entire demand while the decline in her profit per share is infinitely small. Hence, "$a > a^*$" cannot be an equilibrium. Therefore, *a** is the unique Nash equilibrium of the market making game.

The Glosten-Milgrom price has several important properties. Firstly, the expected (half) quoted spread, $\overline{QS} := \mathbb{E}(a^{min} - \tilde{v})$, is strictly positive and just equal to dealers' adverse selection cost. Secondly, the expected (half) realized spread, $\overline{RS} := \mathbb{E}(a^{min} - \tilde{v} \mid \tilde{v}^C > a^{min})$, is equal to zero. The difference between the expected quoted spread and the expected realized spread is that the expected realized spread is computed using realisations of $(a^{min} - \tilde{v})$ (the realized profits of dealers posting the best price) *only* when trades happen $(\tilde{v}^C > a^{min})$.

Thirdly, as shown in figure 2, the expected quoted spread (adverse selection cost) decreases with the variance of the investors' liquidity shocks. Lastly, the Glosten-Milgrom price does not depend on the number of dealers, *N*.

In the case without adverse selection, the client's decision to buy the asset is uninformative since $\tilde{v}^C$ is independent from $\tilde{v}$. Thus, $a^* = \mathbb{E}_\mu(\tilde{v} \mid \tilde{v}^C > a^*) = \mathbb{E}_\mu(\tilde{v})$. Therefore, in this case, the expected quoted spread and realized spread are zero in equilibrium, for all values of the parameters.

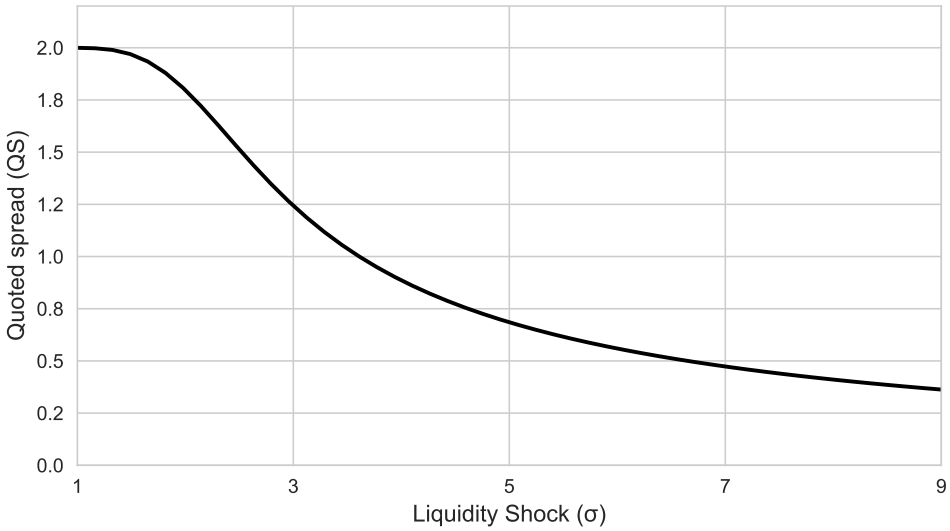


**Figure 2. Glosten-Milgrom Benchmark.** Parameter values are $\mathbb{E}(v)$, $\mu = \frac{1}{2}$, $\Delta(v) = 4$ and $\sigma = 5$. The figure shows the quoted spread in the Glosten-Milgrom Benchmark. The panel shows the quoted spread is a decreasing function of the variance of clients' liquidity shocks, $\sigma$.

# 4 Algorithmic Market Makers

This section describes the problem faced by the Algorithmic Market Makers (AMs) or dealers and how Q-learning and Deep Deterministic Policy Gradient (DDPG) aid the dealers in their pricing strategies.

## 4.1 The Problem

Following Colliard et al. (2026), I now consider the dealers who must play the market making game described in section 3 for $T$ number of "iterations". An iteration consists of only one trading round and realisations of the asset payoffs and clients valuations are independent across iterations. It is assumed that dealers are risk-neutral and only consider the total (non-discounted) profits. Hence, denoting $\pi_{n,t}$ the realized profit of dealer $n$ in iteration $t$, if the dealer were able to form a rational expectation about $\pi_{n,t}$, then she would look for a pricing policy from $t = 1$ to $t = T$ that maximises her total expected profit, that is

$$\mathbb{E}\left[\sum_{t=1}^{T} \pi_{n,t}\right]. \tag{7}$$

Since the market making game is finitely repeated and has a unique Nash-equilibrium, a fully rational dealer would play the Glosten-Milgrom price at each iteration. However, this assumption of fully rationality and complete information is unlikely to hold in practice — real market makers operate in complex environments where the distribution of asset values, the behaviour of clients,

and the strategies of competitors are not directly observable. To capture this more realistic setting, it is assumed that dealers have no prior knowledge of the primitives of the game. At date 0, each dealer $n$ only knows that she will have to select a price in some set $\mathcal{A}$ for each of the $T$ iterations, and she will only observe her realized profit $\pi_{n,t}$ at the end of each iteration $t$. She knows, neither the structure of the trading environment, nor that of the competition, nor, generally, the stochastic mapping from the prices she sets to the payoffs she receives. Thus, each dealer is unable to compute the expectation eq. (7). Given this information or lack thereof, from the perspective of the dealer, choosing prices amounts to a multi-armed bandit problem ; the dealer can try different prices or "arms" to estimate the average payoff associated with each price.

It is assumed that she approaches this problem using a reinforcement learning algorithm. While there are various types of reinforcement algorithms, they all share a common concept: decision makers learn to optimise their behaviour through experimentation. In our context, the experiment involves trying a price, observing a resulting profit, updating the estimates of the profit associated with the price, and then iterating further.

Over time, the experimentation allows the dealer to refine her estimate of the average payoff corresponding to each price. However, it also introduces a fundamental trade-off problem between experimentation and exploitation. Exploitation involves selecting the price with the highest estimated payoff known as "the greedy price". Experimentation, by contrast, involves randomly selecting a price from the available set $\mathcal{A}$ with probability $\varepsilon_t = e^{-\beta t}$, regardless of its estimated payoff, in order to gather information about prices that may not have been recently visited. This strategy of selecting the greedy price with probability $1 - \varepsilon_t$ and a random price with probability $\varepsilon_t$ is known as the $\varepsilon$-greedy policy (Sutton, Barto, et al. 1998).

## 4.2 Q-Learning Algorithms

Let us start with one of the simplest forms of reinforcement learning for our analysis. I assume that the dealers use Q-learning algorithms (Watkins and Dayan 1992). Q-learning is the foundation for more sophisticated algorithms like Deep Deterministic Policy Gradient (DDPG) (Lillicrap et al. 2019) that is described in section 4.3 to solve *n*-armed bandit problems.

The dealers are referred to as Algorithmic Market Makers (AMs). The AMs are restricted to quote from a discrete set of prices in $\mathcal{A} = \{a_1, a_2, \ldots, a_M\}$, where each possible $a_m$ is a possible ask price.[1] This price grid is chosen such that the expected payoff of the asset, the Glosten-Milgrom price, and the monopoly price (the one maximising $\Pi(a, \overline{a}; \mu)$ when $N = 1$) are all in range of $[a_1, a_M]$.

1. This restriction of a discrete set of prices is necessary for Q-learning algorithms. Thus, the price set cannot be continuous

The Q-learning algorithm used by each AM is as follows. To each AM $n$ and iteration $t$, a so-called *Q-matrix* $\mathbf{Q}_{n,t} \in \mathbb{R}^{M\times 1}$ is associated, which is a column vector of size $M$.[2] The $m^{th}$ entry of the matrix, denoted $q_{m,n,t}$, represents the estimate by the $n^{th}$ AM (represented by $AM_n$), in iteration $t$, of the profit from playing price $a_m$. Q-learning algorithms may well start with a clean slate, such as $\mathbf{Q}_0 = 0$. However, the learning process is initialised by assigning each AM an initial Q-matrix $\mathbf{Q}_{n,0}$ with random values drawn from a uniform distribution over $[\underline{q}, \overline{q}]$, i.i.d. across prices and dealers, following Colliard et al. (2026). As Sutton, Barto, et al. (1998, p. 39) note, the initial action-value estimates provide an easy way to supply prior knowledge about the level of rewards that can be expected and can be used as a simple way of encouraging exploration. When initial Q-values are set above the true expected payoffs, whichever price is initially selected will yield a reward below the starting estimate, causing the AM to switch to other prices. The result is that all prices are tried several times before the value estimates converge, encouraging broad exploration of the price space in the early stages of training. In the present model, random initialisation over $[\underline{q}, \overline{q}]$ serves this purpose by assigning different initial Q-values to different prices and across dealers, the AMs are encouraged to explore broadly before their experimentation rate decays.

According to Colliard et al. (2026), for the market making game in Section 3, the Q-learning algorithm specifies i) how an AM chooses her price in iteration $t$, and ii) how an AM's Q-matrix updates over time which reflects the realized payoff given the prices chosen in a given iteration. This specification relies on two parameters (common to all AMs): (i) the learning coefficient, $\alpha \in (0, 1)$ and (ii) the experimentation rate, $\beta > 0$, which determines the probability $\varepsilon_t := e^{-\beta t}$. Parameters $\alpha$ and $\beta$ are fixed (also called "hyper-parameters"). Parameter $\beta$ controls the speed at which $\varepsilon_t$ decays over time. A larger $\beta$ means that $\varepsilon_t$ decays faster and AMs switch more quickly to exploiting. The parameter $\alpha$ controls the sensitivity of the AMs' estimates to new observations. Given this, the following are the three steps for each iteration $t$ between 1 and $T$:

1. **Action:** The behaviour of each AM in iteration $t$ needs to be determined. For each $AM_n$, $m^*_{n,t} := \arg\max_n q_{m,n,t-1}$ is defined and the index associated with the highest value in matrix is $\mathbf{Q}_{n,t-1}$, and denote $a^*_{n,t} := a_{m^*_{n,t}}$, as the *greedy price* of this AM, that is, the price which, according to the AM's estimate yields the largest estimated profit.

   With probability $1 - \varepsilon_t$, $AM_n$ takes an "exploitation" action, that is, she plays the greedy price. With probability $\varepsilon_t$, she takes an "exploration" action: the AM draws a random integer $\tilde{m}_{n,t}$ between 1 and $M$ (all values be equally probable) and quotes $a_{n,t} = a_{\tilde{m}_{n,t}}$. Thus, $a_{\tilde{m}_{n,t}}$ is

2. Generally, the Q-matrix of an agent has $S$ columns, each corresponding to a state realized in each episode that can affect the average payoff obtained by the agent with a certain action. If there is no such state, when $S = 1$, which is the case here, since AMs are not allowed to condition their choice of price on past trading history to be as close as possible to the market making game considered.

chosen randomly from $\mathcal{A}$. Whether to explore and which price to try is drawn independently across AMs, $\overline{a}_t = (a_{1,t}, a_{2,t}, \ldots, a_{n,t})$ denotes the vector of prices quoted by all AMs in iteration $t$ and $a_t^{min} = \min_n\{a_{n,t}\}$ the best offer in iteration $t$.

2. **Feedback:** Then the realized profit for each algorithmic market maker according to the market making framework outlined in Section 3 is calculated. The asset's payoff $\tilde{v}_t$, the client's liquidity shock $\tilde{L}_t$ is drawn, as described in section 3.1. Then the client's valuation is determined, $v_t^C$ as described in the section. If $v_t^C \geq a_t^{min}$, the trade price $a_t^{min}$ is recorded, and otherwise the absence of trade is recorded. In either case, AMs receive a profit equal to $\pi_{n,t} = \Pi(a_{n,t}, \overline{a}_{n,t}, v_t^C, v_t)$, as given by (2). Particularly, AMs quoting $a_t^{min}$ share the profit (or loss) from selling the asset, while others get zero. Moreover, if no trade takes place, then all dealers receive a profit of zero.
3. **Update:** Each AM updates its Q-matrix as follows as given by Colliard et al. (2026)

$$q_{m,n,t} = \begin{cases} \alpha\pi_{n,t} + (1-\alpha)q_{m,n,t-1} & \text{if } a_{n,t} = a_m \\ q_{m,n,t-1} & \text{if } a_{n,t} \neq a_m \end{cases} \tag{8}$$

After playing an action $m$, the AM updates the associated value in the Q-matrix according to two cases.

   a. **Case 1: When price is chosen** (if $a_{n,t} = a_m$): The new Q-value is the $\alpha$ times current period's profit $\pi_{n,t}$ plus $(1-\alpha)$ times the Q-value in time $t-1$, where $\alpha$ is the learning rate.
   b. **Case 2: When price is not chosen** (if $a_{n,t} \neq a_m$): The Q-value $q_{m,n,t}$ remains the same as previous period $q_{m,n,t-1}$ since the AM receives a profit of zero (client does not buy the asset).

### 4.3 Deep Deterministic Policy Gradient (DDPG) Algorithms

Another form of reinforcement learning which can be used for analysis that will be discussed is Deep Deterministic Policy Gradient Algorithms (DDPG) (Lillicrap et al. 2019) as an alternative pricing strategy for algorithmic market makers.

Why would the dealers use DDPG? The DDPG Algorithm enables the dealers to operate in continuous state and action spaces. The algorithm naturally accommodates this market making game by allowing the dealers to select any price within specific bounds, rather than being confined within a predetermined price grid. Another reason to use DDPG is the inability of Q-learning to handle high dimensionality and highly complex action spaces. In our task of interest, this isn't particularly important since the market making game is simply i.i.d, with each state being an independent game, however, in real life where seconds matter in high frequency trading (Hasbrouck and Saar 2013),

DDPG's actor-critic infrastructure enables agents to learn effective strategies with fewer market interactions than required for Q-learning algorithms.

The description for the algorithm is as follows: The DDPG algorithm is based on actor-critic architecture consisting of two neural networks working in tandem. The actor network, parameterized by $\theta^{\varphi}$, approximates the optimal policy function $\varphi^*(s_t)$. Unlike the discrete Q-matrix $Q_{n,t} \in \mathbb{R}^{M\times 1}$ used in Q-learning, the critic network, parameterized by $\theta^{Q}$, approximates the optimal state-action value function $Q^*(s_t, a_t)$, which evaluates the expected profit $\pi_{n,t}$ for executing action $a_t$ in market state $s_t$ during iteration $t$. In order to adapt Q-learning algorithms to make use of large neural networks as function approximators Mnih et al. (2013) introduced two major changes which were later adapted by Lillicrap et al. (2019) and these are as follows:

1. **Replay buffer:** A replay buffer is a finite sized cache $\mathcal{D}$ which stores the transition tuple $(s_t, a_t, r_t, s_{t+1})$. When the replay buffer is full the oldest samples are discarded.
2. **Target Network:** Directly implementing Q-Learning using neural networks has proven to be unstable. Since the network $Q(s, a|\theta^{Q})$ is being updated and also used to calculate the target value. Lillicrap et al. (2019) propose creating a copy of the actor and critic networks denoted by $\theta^{\varphi\prime}$ and $\theta^{Q\prime}$ respectively, that are used for calculating target values. It may slow learning, however, having both $\varphi'$ and $Q'$ was required to have stable learning.

Gufler et al. (2025) provide an extensive explanation of the framework of the DDPG algorithm. Let $|B|$ denote a random sample of a batch of transitions from the replay buffer $\mathcal{D}$.

The critic network predicts Q-values, i.e., future expected reward. It minimises the loss function between the target and the predicted Q-values by

$$L(\theta^{Q}) = \mathbb{E}\left[(Q(s_t, a_t|\theta^{Q}) - y_i)^2\right] \tag{9}$$

where the target Q-value $y_t$ is set by

$$y_i = r_i + \gamma Q'(s_{i+1}, \varphi'(s_{i+1}|\theta^{\varphi\prime})|\theta^{Q\prime}), \quad i \in B$$

However, the original DDPG algorithm was designed for tasks where future rewards depend on current action using a discount factor $\gamma \in [0, 1]$ to weight future returns. In our market making game, each interaction between the AM and the client is independent. Since no state transitions depend on past actions, the target $y_i$ is reduced to

$$y_i = r_i, \quad i \in B \tag{10}$$

which sets $\gamma = 0$, eliminating any temporal difference component from the critic update. The critic therefore learns a pure profit estimation rather than a discounted value function. The critic parameters are updated using a gradient descent on the loss function eq 9 at learning rate $\eta_Q$,

$$\theta^Q \leftarrow \theta^Q - \eta_Q \nabla_{\theta^Q} L(\theta^Q), \tag{11}$$

where the gradient is,

$$\nabla_{\theta^Q} L(\theta^Q) = \frac{1}{|B|} \sum_{i \in B} \left( Q(s_t, a_t | \theta^Q) - y_i \right) \nabla_{\theta^Q} Q(s_t, a_t | \theta^Q). \tag{12}$$

The behaviour of $L(\theta^Q)$ over the course of training serves as a diagnostic indicator of convergence. As the critic observes more transitions from the replay buffer $\mathcal{D}$, it produces increasingly accurate Q-value estimates, reflected in a declining critic loss. Theoretically, if the critic loss fails to decline or diverges, the actor receives unreliable gradient signals and cannot learn a meaningful pricing strategy, since the actor update direction $\nabla_{\theta^\varphi} J$ depends entirely on $\nabla_a Q$ — the gradient of the critic's Q-value with respect to the action.

On the other hand, the actor network $\varphi(s|\theta^\varphi)$ is updated by gradient ascent since the expected profit needs to be maximised rather than minimising a loss. The objective function for the actor is:

$$J(\theta^\varphi) = \mathbb{E}\left[ Q(s, a | \theta^Q)|_{a=\varphi(s|\theta^\varphi)} \right]. \tag{13}$$

The actor is updated by applying the chain rule, the policy gradient theorem (Silver et al. 2014) is as follows:

$$\nabla_{\theta^\varphi} J(\theta^\varphi) \approx \frac{1}{|B|} \sum_{i \in B} \nabla_a Q(s, a | \theta^Q)|_{s=s_i, a=\varphi(s_i|\theta^\varphi)} \cdot \nabla_{\theta^\varphi} \varphi(s|\theta^\varphi)|_{s=s_i} \tag{14}$$

Similar to the critic loss, the actor loss $-\frac{1}{|B|} \sum_{i \in B} Q(s_i, \varphi(s_i|\theta^\varphi)|\theta^Q)$ serves as a convergence diagnostic for the actor. Unlike supervised learning loss which converges to zero, the actor loss converges to a negative plateau reflecting the stable Q-values of the learned pricing strategy, since it is defined as the negative mean Q-value of the actor's chosen actions. A stabilising actor loss therefore indicates the actor has converged to a consistent policy and is no longer finding significantly more profitable pricing actions. Together, a declining critic loss and a stabilising actor loss provide joint evidence that both networks have converged to a stable equilibrium, justifying the reporting of exploitation phase prices as genuine learned strategies rather than unstable behaviour.

The gradient is split into two parts: $\nabla_a Q(s, a|\theta^Q)$, which captures the sensitivity of the critic's Q-value prediction to changes in action, and $\nabla_{\theta^\varphi} \varphi(s|\theta^\varphi)$ which records how the actor's policy responds to changes in its parameters. Simply, these two effects quantify for the actor in which direction to

change the price to increase the critic's estimated profit and how to change its network weights to produce that price. The actor parameters are updated by gradient ascent at learning rate $\eta_\varphi$, given by,

$$\theta^\varphi \leftarrow \theta^\varphi + \eta_\varphi \nabla_{\theta^\varphi} J(\theta^\varphi) \tag{15}$$

Finally, the target networks are updated via a soft update rule that slowly updates it into the current networks. After each step, target networks are updated according to:

$$\theta^{Q\prime} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q\prime}$$
$$\theta^{\varphi\prime} \leftarrow \tau\theta^\varphi + (1-\tau)\theta^{\varphi\prime}$$

where $\tau \ll 1$ is the update rate, and it ensures the target network tracks the learned networks slowly, providing stable values during the training process.

# 5 Experimental Design

This section explains the methodology of the algorithms and how they are applied to the Market Making Game.

## 5.1 Q-Learning

The experimental design of Q-learning is based on the framework by Colliard et al. (2026). For a given parametrization of the market making game $(\Delta_v, \sigma, \mathbb{E}_\mu(\tilde{v}))$, different runs of $T_Q$ iterations can lead to different outcomes, even when starting from the same deterministic initial Q-matrix. In fact, the profit $\pi_{n,t}$ the dealer $n$ receives after each iteration $t \in \{1, \ldots, T_Q\}$ with a given price is stochastic. It depends on the realisation of the asset payoff in the iteration, $\tilde{v}_t$, and the client's valuation, $\tilde{v}_t^C$. Therefore, for a given history the AM can be "lucky" with a certain price and end up choosing that price very often, whereas, in another history, the same AM can be unlucky with this same price and plays differently. Moreover, since the exploration path of randomly chosen prices differs across experiments due to the stochastic nature of the $\varepsilon$-greedy strategy, two experiments starting from the same initial Q-matrix may end up at different long-run greedy prices even under identical market parameters. To address this issue Colliard et al. (2026) suggest running a large number of *K* experiments of $T_Q$ iterations each, holding the parameters of the market making game constant, and after the experiments comparing the distribution of outcomes (for example, the average and standard deviation of quoted spreads) across experiments.

Colliard et al. (2026) address the non-convergence to a constant action of Q-learning algorithms by analysing distributional stability over very long horizons, that is, if $T_Q$ is large enough then the distribution of the outcomes has converged. However, following established algorithmic pricing literature (Calvano et al. 2020), I implement a different convergence detection mechanism to distinguish between the learning and exploitation phases of Q-learning. This "off-line" approach reflects the realistic assumption that the AMs train the algorithms off-line before being put to work as often the case in financial markets where algorithms are calibrated and tested in a controlled environment prior to live trading (European Securities and Markets Authority (ESMA) 2026, p. 6). This approach is valid here since the market making game is stationary as the underlying market parameters do not change over time meaning the strategy learned during training remains optimal while deployed.

Formally, the convergence criterion is as follows: convergence is deemed to have been achieved if for each dealer the optimal strategy does not change for 50,000 consecutive iterations. That is, if for each $AM_n$ in iteration $t$ the index $m^*_{n,t} := \arg\max_n q_{m,n,t-1}$, remains constant for 50,000 iterations, it is assumed that algorithms have completed the learning process and attained stable behaviour.[3] This convergence criterion is an important factor to ensure a fair comparison between the two algorithms. The number of training iterations for the DDPG algorithm is calibrated to the mean convergence iteration $\bar{T}^{conv}$ of the Q-learning algorithm across $K$ experiments rounded to the nearest zero. This approach ensures that both algorithms are allocated a comparable computational budget that reflects the number of market interactions Q-learning requires to converge to a stable pricing strategy, rather than imposing a strict budget that favours one algorithm over the other. This choice is further explained in Section 5.2

Based on Colliard et al. (2026), the parameters of the market making game are as follows: $\Delta_v = 4, \sigma = 5, v_H = 4, v_L = 0, \mu = 0.5,$ and $N = 2$ (same parameters as figure 1). In addition to that, AMs can choose prices from a discrete grid between 1.1 and 14.9 with a tick size of 0.1 (139 prices in total). This specification of prices makes sure that the Glosten-Milgrom prices are in range of the possible prices in all specifications. The Q-matrices are initialised with random values following a uniform distribution between $\underline{q} = 3$ and $\overline{q} = 6$, so that all initial Q-values are above a maximal payoff a dealer can get in a given period.[4]

Given this, I run $K = 10$ experiments, each with $T_Q = 500,000$ iterations where one iteration is one independent market interaction for training the algorithm. After training is completed based

3. Note that even if $\arg\max_n q_{m,n,t-1}$ does not change, the Q-matrices themselves might keep changing. Alternatively, standard of convergence would require that the entries of Q-matrics not change by more than x% for certain number of periods. (Calvano et al. 2020)

4. Q-values are chosen in such a way to guarantee that all actions are chosen sufficiently to overcome the initial values of the Q-matrix (See Asker et al. 2024)

on the convergence criterion, $\tilde{T} = 10,000$ iterations are run in pure exploitation mode without any added exploration noise to obtain final statistics for comparison analysis. In all experiments $\alpha = 0.01$ and $\beta = 8 \cdot 10^{-5}$. Following the framework of Colliard et al. (2026), for each experiment $k \in \{1, \ldots, K\}$ and exploitation iteration $t \in \{1, \ldots, \tilde{T}\}$, the minimum ask price $a_t^{min,k}$ and the realised asset value $v_t^k$ are recorded. The per-experiment quoted spread $QS^k$ and realised spread $RS^k$ are computed over the exploitation phase of $\tilde{T} = 10,000$ iterations as:

$$QS^k = \frac{1}{\tilde{T}} \sum_{t=1}^{\tilde{T}} \left( a_t^{min,k} - \mathbb{E}[\tilde{v}] \right), \qquad RS^k = \frac{1}{\tilde{T}} \sum_{t=1}^{\tilde{T}} \left( a_t^{min,k} - v_t^k \right) \tag{16}$$

The per-experiment spreads $QS^k$ and $RS^k$ are then averaged across $K$ experiments to obtain:

$$\overline{QS} = \frac{1}{K} \sum_{k=1}^{K} QS^k, \qquad \overline{RS} = \frac{1}{K} \sum_{k=1}^{K} RS^k \tag{17}$$

## 5.2 Deep Deterministic Policy Gradient

The DDPG algorithm begins by initialising actor and critic parameters with random weights $\theta^{\varphi}$ and $\theta^{Q}$ respectively, alongside the target network counterparts $\theta^{\varphi\prime}$ and $\theta^{Q\prime}$ which are set equal to the main network at initialisation. An empty replay buffer $\mathcal{D}$ with capacity $10^5$ is also initialised to store transition experiences.

At each step *t*, the dealer observes the current asset value $v_t \in \{v_H, v_L\}$ and selects an action price by passing the state through the actor network and adding noise for exploration:

$$a_t = \varphi(s_t|\theta^{\varphi}) + \mathcal{N}(0, \sigma_k^2) \tag{18}$$

The exploration noise standard deviation decays exponentially from $\sigma_{\text{initial}} = 0.5$ to $\sigma_{\text{final}} = 0.01$ over 90% of the total $T_D$ iterations, ensuring the AM transitions gradually from broad exploration of the continuous price space to a near-deterministic policy in the final 10% of iterations. Setting the decay horizon strictly below the total number of training iterations reserves a consolidation period during which the policy stabilises without the distortion of exploration noise, ensuring that the reported exploitation phase prices reflect a converged rather than still-exploring strategy. The resulting price is clipped to the action bounds which are continuous in nature $[a_{min}, a_{max}] = [1.1, 14.339]$.

After executing the quoted price, the $AM_n$ observes the resulting profit $r_t$, stores the transition tuple $(s_t, a_t, r_t, s_{t+1})$ in the replay buffer $\mathcal{D}$. Once the buffer contains at least $|B| = 500$ transitions, the networks begin updating at each step by sampling a random minibatch $B$ from $\mathcal{D}$.

The critic is updated by minimising the mean squared error between its predicted Q-values and the targets over the minibatch. The actor is then updated by gradient ascent, using the critic's gradient to determine the direction in which to adjust quoted prices to increase expected profit.

Finally, the target networks are updated via soft updates with $\tau = 0.001$, ensuring they track the main networks slowly and providing stable target values throughout training. Both actor and critic networks consist of two hidden layers with 64 units and 32 units respectively. I use the Adam optimiser (Kingma and Ba 2014; Lillicrap et al. 2019) for learning the neural network parameters with learning rate $\eta_\varphi = 10^{-4}$ for the actor network and $\eta_Q = 10^{-3}$ for the critic network. Full summary of hyperparameters is provided in the table 1 below.

**Table 1.** DDPG Hyperparameters

| | | |
|---|---|---|
| $\eta_\varphi$ | $10^{-4}$ | Actor Learning rate |
| $\eta_Q$ | $10^{-3}$ | Critic Learning rate |
| $\tau$ | $10^{-3}$ | Soft update rate |
| $\lvert\mathcal{D}\rvert$ | $10^5$ | Replay buffer capacity |
| $\lvert B\rvert$ | 500 | Minibatch size |

The DDPG algorithm is trained over a total of $T_D = 154,000$ iterations per experiment where one iteration is one market interaction. The choice of $T_D$ is calibrated on the empirical convergence behaviour of the Q-learning algorithm. Specifically, the Q-learning algorithm is first run for $K = 10$ and the mean convergence iteration $\bar{T}_Q^{conv}$ is recorded. The total number of training iterations is then set to $T_D = \bar{T}_Q^{conv}$. The convergence statistics of the Q-learning algorithm across experiments is reported in Table 2 in Section 7.2. Following the same convention as the Q-learning setup, I run $K = 10$ independent experiments, each initialised with a different random seed, to obtain training results. Since the underlying game is i.i.d., each of the 154,000 market interactions within an experiment are statistically independent, analogous to the 500,000 iterations in the Q-learning[5] framework. After the training phase ends, I run additionally a pure exploitation phase consisting of $\tilde{T} = 10,000$ iterations for final comparison analysis. Following the same measurement convention as Q-learning, for each experiment $k \in \{1, \ldots, K\}$ and exploitation iteration $t \in \{1, \ldots, \tilde{T}\}$, the minimum quoted price $a_t^{min,k}$ and the realised asset value $v_t^k$ are recorded. The per-experiment quoted spread $QS^k$ and realised spread $RS^k$ are computed over the exploitation phase of $\tilde{T} = 10,000$ iterations as:

$$QS^k = \frac{1}{\tilde{T}} \sum_{t=1}^{\tilde{T}} \left( a_t^{min,k} - \mathbb{E}[\tilde{v}] \right), \qquad RS^k = \frac{1}{\tilde{T}} \sum_{t=1}^{\tilde{T}} \left( a_t^{min,k} - v_t^k \right) \tag{19}$$

5. Q-learning algorithm may converge before hitting the 500,000 iteration mark. Therefore, the effective training iterations is lower.

The per-experiment spreads $QS^k$ and $RS^k$ are then averaged across $K$ experiments to obtain:

$$\overline{QS} = \frac{1}{K}\sum_{k=1}^{K} QS^k, \qquad \overline{RS} = \frac{1}{K}\sum_{k=1}^{K} RS^k \tag{20}$$

Given below is the **pseudocode** of the algorithm for better understanding,

**Algorithm 1** Deep Deterministic Policy Gradient

**Input:** initial actor parameters $\theta^{\varphi}$, critic parameters $\theta^{Q}$, empty replay buffer $\mathcal{D}$
Set target network parameters equal to main parameters: $\theta^{Q'} \leftarrow \theta^{Q}$, $\theta^{\varphi'} \leftarrow \theta^{\varphi}$
**repeat**
    Observe asset value $s_t$ and select action: $a_t = \text{clip}(\varphi(s_t|\theta^{\varphi}) + \varepsilon, a_{min}, a_{max}),$ where $\varepsilon \sim \mathcal{N}$
    Execute $a_t$ in the environment
    Observe reward $r_t$ and next state $s_{t+1}$
    Store $(s_t, a_t, r_t, s_{t+1})$ in replay buffer $\mathcal{D}$
    **if** it is time to update **then**
        **for** however many updates **do**
            Randomly sample a minibatch of transitions $B = \{(s_i, a_i, r_i, s_{i+1})\}$ from $\mathcal{D}$
            Compute targets:

$$y_i = r_i, \quad i \in B$$

            Update critic by one step of gradient descent using:

$$\nabla_{\theta^{Q}} \frac{1}{|B|} \sum_{i \in B} \Big( Q(s_i, a_i|\theta^{Q}) - y_i \Big)^2$$

            Update actor by one step of gradient ascent using:

$$\nabla_{\theta^{\varphi}} \frac{1}{|B|} \sum_{i \in B} Q(s_i, \varphi(s_i|\theta^{\varphi})|\theta^{Q})$$

            Update target networks:

$$\theta^{Q'} \leftarrow \tau\theta^{Q} + (1-\tau)\theta^{Q'}$$

$$\theta^{\varphi'} \leftarrow \tau\theta^{\varphi} + (1-\tau)\theta^{\varphi'}$$

        **end for**
    **end if**
**until** convergence

# 6 Hyperparameter Selection

In this section, I justify the choice of hyperparameters for both Q-learning and DDPG algorithms.

The hyperparameters for the Q-learning algorithm are selected following extensive experimentation in established literature of the market making game. Generally, the learning parameter $\alpha$

ranges from 0 to 1. However, extreme values of $\alpha$ tend to be disruptive to learning. When $\alpha = 0$, the algorithm does not learn at all; when $\alpha = 1$, the algorithm immediately forgets what it has learned in the past. So, learning is expected to be most effective for intermediate values of $\alpha$.

Calvano et al. (2020) provide reasoning and trade-offs between selection of different values of $\alpha$ and $\beta$. Colliard et al. (2026) conduct an extensive analysis of hyperparameter sensitivity for Q-learning in the context of the market making game, examining the effect of varying $\alpha$ and $\beta$ on the expected profit outcomes of both dealers. The intuition is that while higher learning rate $\alpha$ and lower experimentation rate $\beta$ can boost short-term profit by undercutting opponents, the long-term effect is negative. Indeed, once $AM_1$ undercuts $AM_2$, $AM_2$ makes no profit and eventually adjusts, forcing $AM_1$ to share demand at lower prices. This long-term effect outweighs short-term gains.In sum, they find no support for the idea that competition on the choice of AM's hyperparameters should fix AM's failure to learn to be competitive. Therefore, the thesis adopts the baseline parameterisation of Colliard et al. (2026), ensuring compatibility with their findings. The literature on this is still very scarce and remains an open research question, as noted by Colliard et al. (2026).

For the DDPG algorithm, the hyperparameters follow the recommendations of Kingma and Ba (2014) and Lillicrap et al. (2019), who demonstrate separate learning rates $\eta_\varphi = 10^{-4}$ and $\eta_Q = 10^{-3}$ for actor and critic networks respectively, produce stable convergence across the range of tasks. The soft update rate $\tau = 10^{-2}$ is similarly adopted from the original paper. While the original paper of Lillicrap et al. (2019) employs a replay buffer of capacity $10^6$, two hidden layers of 400 and 300 units for actor and critic networks respectively, and a minibatch size of 64, the present thesis adapts these choices to reflect the considerably simpler nature of the market making game. Accordingly, the actor and critic networks employ two hidden layers of 64 units and 32 units respectively, which is sufficient to approximate optimal outcomes in the low-dimensional environment. The replay buffer capacity is reduced to $|\mathcal{D}| = 10^5$ and is set slightly below the total number of training iterations $T_D = 154{,}000$. Since the market making game is i.i.d., transitions are already statistically independent and the primary function of the replay buffer is to provide stable minibatch estimates. Setting the buffer capacity to a lower amount ensures early transitions under higher noise environments are discarded as training progresses, so that network updates in latter stages are drawn from converged low-noise policy. This choice is well suited to the structure of the game and does not affect the stability or quality of learning.

# 7 Experimental Findings

In this section I describe the Algorithmic Market Makers' behaviour and present the experimental findings from both Q-learning and DDPG with the comparative analysis between the two methods. Before presenting the main exploitation phase results, I first examine the training dynamics from the exploration phase of both algorithms for agent 1[6]. This serves two purposes: first, it provides evidence that both algorithms have converged to a stable pricing strategy by the end of the training phase, validating the exploitation phase prices as the primary outcome measure. Second, it illustrates the fundamental difference in how the algorithms learn, which directly correlates to the differences reported in the exploitation phase results in Section 7.2.

## 7.1 Training Dynamics

The summary of convergence statistics for the Q-learning agent is reported in Table 2. After running the Q-learning algorithm for $K = 10$ experiments each with a maximum of $T_Q = 500,000$ iterations, the mean convergence iterations across experiments is $\bar{T}^{conv} = 154,288$ with a standard deviation of $\sigma^{conv} = 20,177$. Across all ten experiments, the Q-learning AM converges 100% of the time.

**Table 2.** Q-Learning Convergence Statistics

| Metric | Value |
|---|---|
| Mean Convergence Iterations | 154,288 |
| Std Convergence Iterations | 20,177 |
| Convergence Rate | 100% |

The actor and critic loss functions of the DDPG algorithm, which provide evidence of convergence, are presented in Appendix A.2.

Figure 3 shows the distribution of greedy prices during the training phase for Q-learning (left) and DDPG (right). Each histogram pools all recorded greedy-price snapshots across all training iterations and all experiments. For Q-learning, the greedy price is recorded once every 100 iterations over a maximum of $T_Q = 500{,}000$ iterations, yielding up to 5,000 snapshots per experiment and up to 50,000 data points in total. In experiments where both agents' greedy actions stabilise for 50,000 consecutive iterations before $T_Q$ is reached, training ends early and fewer snapshots are

6. Although $AM_1$ and $AM_2$ follow different training trajectories due to independent random initialisation and stochastic exploration paths, both agents converge to virtually identical exploitation phase prices in all experiments, consistent with the symmetric structure of the market making game. Only $AM_1$ is presented in the figures throughout this section. The training dynamics and exploitation phase prices of $AM_2$ are qualitatively identical and are provided in Appendix A.1 for completeness.

contributed. For DDPG, one greedy-price snapshot is recorded every 100 iterations, yielding exactly 1,540 snapshots per run and 15,400 data points in total across all 10 runs. The distribution therefore captures the full trajectory of learning, not just the final converged price, so the mean, median, and standard deviation reflect both how prices evolved during training and where different experiments ultimately settled. The left panel of the figure shows that the Q-learning AM's greedy-price is widely dispersed throughout training. The mean greedy price is 5.89 and median greedy price is 4.8 and a standard deviation of 2.74 reflects the slow drift of prices over training iterations through the trial-and-error method of Q-learning. In all experiments, the greedy price is above the Glosten-Milgrom price ($a^*$ = 2.69) and the least competitive Nash-equilibrium price 2.8 (about one tick above the Glosten-Milgrom price). At any price, above 2.8, in theory, an AM can obtain a strictly larger profit by undercutting her competitor. For example, if both the AMs settle on the price of 5. At this price, in the baseline case, each AM obtains a true expected profit of 0.30. However, each AM could obtain a larger profit, of 0.59, simply by undercutting her competitor by posting a price of 4.90. However, by posting a price of 4.90, the AM invites her competitor to post a lower price, say 4.90, resulting in both AMs being worse off. Nevertheless, the design of AMs in Q-learning does not allow them this forward-looking reasoning, in particular, since AMs cannot condition their prices on past trading history, they cannot learn that undercutting might generate a loss in future profits by triggering a drop in her competitor's price.

The right panel of the figure shows a different pattern for the DDPG AM. The mean of the greedy-price distribution is 3.18 and median of the distribution is 3.11. The AM's quotes vary across experiments with a standard deviation of 0.44. The narrow spread indicates that DDPG converges to a similar price reliably across all 10 experiments, and does so throughout training. Unlike Q-learning, the AM using DDPG successfully learns to undercut her competitor, achieving a mean greedy price which closely approximates the Glosten-Milgrom benchmark price ($a^*$ = 2.69) and Nash equilibrium price ($a^*$= 2.8) by operating in continuous action and state spaces and through a critic that captures long-run consequences of pricing decisions.

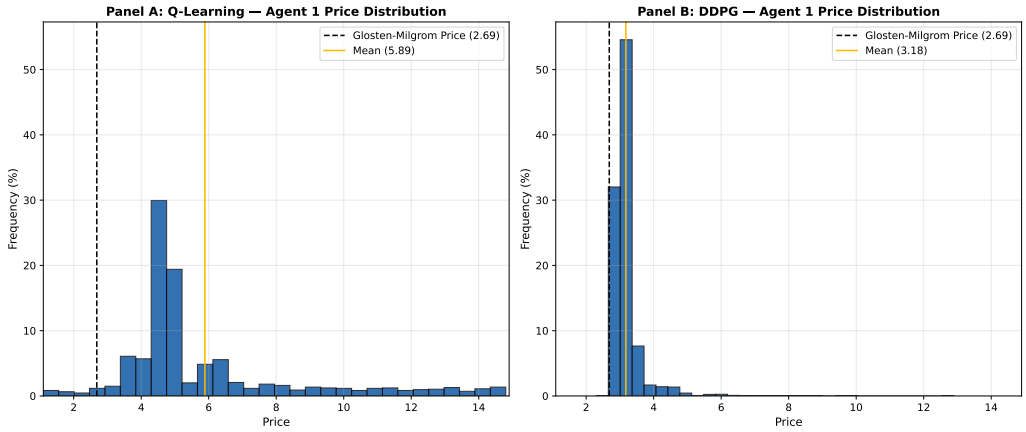


**Figure 3. Distribution of the greedy price of $AM_1$ in adverse selection case.** Parameters: $\sigma = 5, \Delta v = 4, N = 2, \mu = \frac{1}{2}, \mathbb{E}(v) = 2, T_Q = 500,000, T_D = 154,000, \tilde{T} = 10,000, K = 10$. This figure shows a histogram of the greedy price of $AM_1$ for Q-learning (left) and DDPG (right). For each possible price $a$ between 1.1 and 14.9 the bar indicates the percentage of 500,000 iterations for Q-learning and 154,000 iterations for DDPG conducted in which $a_1^* = a$.

Panel A of figure 4 shows for a Q-learning AM the price evolution of the average greedy price over $T_Q = 500,000$ iterations and the greedy prices one standard deviation away from the average (greedy prices vary across experiments due to randomness in trading). During the training phase the average greedy price decreases. In the early phase of the learning process, AMs learn to reduce their price to attract the client and increase their profit share. However, over time, the greedy price declines less to stabilise at a price (4.88 on average across experiments in the training phase) above the competitive price.

Panel B of figure 4 shows for a DDPG AM, the price evolution of the average greedy price over $T_D = 154,000$ iterations and the greedy prices one standard deviation away from the average. The start of the training phase is noisier by design for actor-critic architecture which is represented by the larger standard deviation bands for the first approximately 15000 training iterations. Over the rest of the training period, the mean greedy price gradually stabilises as the exploration noise decays, converging to a mean price of $a^* = 3.28$. Notably, an increase in the standard deviation band is observed between iterations 70,000 and 120,000, which coincides with the exploration noise approaching its minimum value of $\sigma_{\text{final}} = 0.01$ towards the end of the training period. During this period, small disturbances in the actor network weights are no longer masked by exploration noise, causing temporary dispersion in quoted prices across experiments before the policy fully stabilises in the final iterations of training.

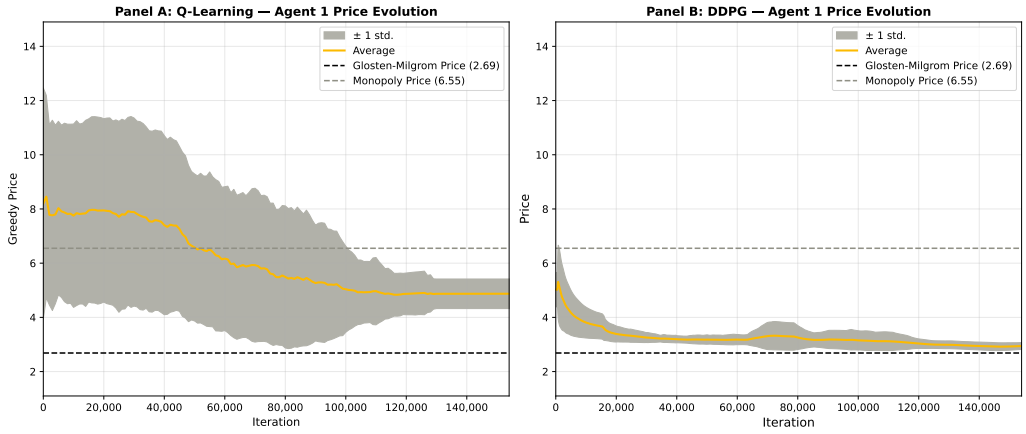


**Figure 4. Price evolution of greedy price of $AM_1$ in adverse selection case.** Parameters: $\sigma = 5$, $\Delta v = 4$, $N = 2$, $\mu = \frac{1}{2}$, $\mathbb{E}(v) = 2$, $T_Q = 500,000$, $T_D = 154,000$, $\tilde{T} = 10,000$, $K = 10$. The left panel of this figure shows the price evolution of the Q-learning agent for each iteration $t$ between 1 and 500,000 the average of $AM_1$'s greedy price $a^*_{1,t}$. The right panel of this figure shows the price evolution of the DDPG agent for each iteration $t$ between 1 and 154,000 the average greedy price $a^*_{1,t}$ of $AM_1$. The solid line represents a 10% moving average for better readability. As a measure of dispersion, standard deviation of $a^*_1$ across experiments is computed and the average of $a^*_1$ plus/minus one standard deviation is plotted for both panels.

There is a stark contrast in the price evolution between the two algorithms. The Q-learning algorithm exhibits considerable volatility across experiments, which is consistent with its purely stochastic exploration mechanism — at each iteration the agent explores with probability $\varepsilon_t = e^{-\beta t}$, leading to high variance in quoted prices particularly in the early stages of learning. The DDPG algorithm, by contrast, displays a more structured convergence pattern driven by its exponentially decaying exploration noise and actor-critic architecture, which provides a stable gradient signal throughout training. Despite this difference in convergence behaviour, both algorithms ultimately learn to quote prices above the theoretical benchmark, though DDPG does so to a substantially lesser degree. These training dynamics provide the necessary context for interpreting the exploitation phase results in the following section.

## 7.2 Equilibrium Price

The exploitation phase prices and standard deviation prices along with the t-test statistic are evaluated for both algorithms and reported in Table 3. The DDPG algorithm converges to a mean greedy price of $a^*_1 = 2.97$ and $a^*_2 = 2.98$ for Agent 1 and Agent 2 respectively, substantially closer to the Glosten-Milgrom competitive benchmark of $a^* = 2.69$ than the Q-learning algorithm, which converges to a mean greedy price of $a^* = 4.88$ for both agents. The standard deviation of greedy prices is lower for DDPG than for Q-learning, consistent with the greater stability of the actor-critic archi-

tecture relative to the purely stochastic exploration mechanism of Q-learning. To formally assess whether the deviation from the Nash equilibrium benchmark is statistically significant, a one sample $t$-test is conducted for each algorithm with $H_0 : \mu = p^* = 2.69$. For Q-learning, the $t$-statistic of $12.15$ is significant at the 1% level for both agents. For DDPG, the $t$-statistic of $6.16$ and $5.68$ for $AM_1$ and $AM_2$ respectively are also significant[7] at the 1% level. However, the magnitude of the DDPG AM's deviation is considerably smaller, approximately $0.28$ above the benchmark compared to $2.19$ for the Q-learning AM, representing a deviation roughly eight times smaller. This finding is further corroborated when comparing against the results of Colliard et al. (2026), where Q-learning is run under considerably more favourable conditions of $T_Q = 1{,}000{,}000$ episodes and $K = 1{,}000$ experiments. Despite this computational advantage, the Q-learning algorithm still converges to less competitive prices, whereas DDPG achieves a more competitive outcome under a fraction of the computational budget, suggesting that the superiority of DDPG is driven by the algorithm itself rather than the scale of the simulation. The prices set by the Q-learning AM are quite an interesting finding.

**Table 3.** Comparison of Greedy Prices for DDPG and Q-learning AMs in the adverse selection case. Parameters: $\sigma = 5, \Delta v = 4, N = 2, \mu = \frac{1}{2}, \mathbb{E}(v) = 2, T_Q = 500,000\ T_D = 154,000, \tilde{T} = 10,000, K = 10.$

| | GM Benchmark | Colliard et al. (2025) | Q-Learning | DDPG |
|---|---|---|---|---|
| *Panel A: Agent 1 Price* | | | | |
| Mean | 2.69 | 4.97 | 4.88 | 2.97 |
| Std | — | 0.73 | 0.57 | 0.15 |
| $t$-stat | — | — | $12.15^{***}$ | $6.16^{***}$ |
| *Panel B: Agent 2 Price* | | | | |
| Mean | 2.69 | 4.97 | 4.88 | 2.98 |
| Std | — | 0.73 | 0.57 | 0.16 |
| $t$-stat | — | — | $12.15^{***}$ | $5.68^{***}$ |

$^{***}p < 0.01$, $^{**}p < 0.05$, $^{*}p < 0.10$. One-sample $t$-test, $H_0$: exploitation-phase mean = GM benchmark price ($N = 10$ experiments).

The supra-competitive prices set seem to suggest that the Q-learning AMs are colluding without any explicit communication. This result is further examined in Section 8.

## 8 Collusion Among Algorithmic Market Makers

Calvano et al. (2020), Colliard et al. (2026) and Dou et al. (2025) document that Q-learning algorithms exhibit some form of AI collusion. The reason this is an important result in my thesis is that AI col-

7. The statistically significant rejection of $H_0$ given the small number of experiments ($N = 10$), therefore reflects the limited power of the test; even a failure to reject $H_0$ should be interpreted with caution.

lusion has been a concern in financial markets and to test whether a more sophisticated algorithm like DDPG may be employed to avoid collusion among AI traders. AI collusion is described as scenarios where autonomous RL algorithms learn to coordinate behaviour without explicit agreements, direct communication or human intervention. Dou et al. (2025) empirically find that AI collusion in securities trading can robustly arise in two ways: through price-trigger strategies and through over-pruning learning bias. The model described in Dou et al. (2025) is different from the one described in this thesis. However, the intuition behind AI collusion remains intact.

The price-trigger mechanism is ruled out by design in the present model. Since the market making game is i.i.d., each interaction is an independent draw of the asset value and the client's liquidity shock. Therefore, AMs cannot condition their prices on past trading history. There is no mechanism through which one AM can detect and punish a deviation by the other, precluding the emergence of price-trigger collusion. This is consistent with Colliard et al. (2026), who similarly rule out tacit collusion by design in their experimental framework. The collusive behaviour observed in the present model is therefore attributed to collusion through "Artificial Stupidity" as described by Dou et al. (2025) also known as "Over-Pruning Bias".

The intuition for this particular form of AI collusion lies in the asymmetry of the estimated Q-value updates in response to noise trading shocks, an innate feature to RL due to its reliance on exploitation. When a dealer quotes a competitive price and the asset value is realised as $v_t = v_H$, the dealer incurs a loss, causing a sharp downward revision of the Q-value associated with competitive pricing, treating it as a very poor action. This discourages the algorithm from revisiting this strategy during exploitation, thereby locking in the downward bias on its estimated value. Conversely, when a dealer quotes a supra-competitive price and asset value is realised as $v_t = v_L$ or $v_t = v_H$, the algorithm earns positive profits and may initially overestimate the Q-value. However, because exploitation leads to frequent reuse of strategies with high estimated Q-values, the algorithm continually revisits this action, allowing the estimated Q-value to be eventually corrected through sufficient updates. Dou et al. (2025) argue that trading outcomes dominated by random noise leads to the asymmetry in the exploitation process especially pronounced and cannot be corrected through exploration. This asymmetry in Q-value updates leads to competitive prices being prematurely pruned due to occasional large losses, while supra-competitive prices are reinforced due to consistently positive returns causing the Q-learning algorithm to develop a biased value system that systematically favours high prices. As both competing dealers are subject to the same bias, they collectively converge to supra-competitive pricing, producing an implicitly collusive outcome without any explicit coordination.

Dou et al. (2025) also find that aggressive trading strategies, being more exposed to noise trading shocks, are more likely to be prematurely pruned. In the model described in this thesis, the variance

of $\sigma$ governs the severity of this bias. Higher values of $\sigma$ increase the probability of a trade occurring at any price level. This amplifies the likelihood of competitive prices being pruned earlier. Whether the DDPG AM is susceptible to this same bias is examined in the following subsection through a comparative analysis of quoted and realized spreads across different values of $\sigma$.

## 8.1 Clients' liquidity shock

This subsection examines the sensitivity of the quoted spread $\overline{QS}$ and realized spread $\overline{RS}$ to changes in the dispersion of clients' liquidity shocks. If the over-pruning bias is indeed the mechanism driving supra-competitive pricing in Q-learning, then quoted and realized spreads should increase with $\sigma$ for Q-learning which is consistent with the prediction that higher noise amplifies the bias, while DDPG spreads should track the Glosten-Milgrom benchmark regardless of $\sigma$, reflecting its robustness to noisy feedback.

For both algorithms I run $K = 10$ experiments for different values of $\sigma = \{1, 3, 5, 7, 9\}$ (other parameters same as in baseline case), both with and without adverse selection ($200 = 2\times2\times10\times5$ ). Thereafter, for each value of $\sigma$, the average quoted spread and average realized spread are computed over the exploitation phase across all $K$ experiments based on the theory described in section 3.3

I begin by looking at expected quoted spread which is the cost of adverse selection described in Section 3.3 with the variance of the client's liquidity shocks. Figure 5 shows for a Q-learning AM (Panel A) and a DDPG AM (Panel B) the effect of dispersion of clients' liquidity shock $\sigma$ on the AMs' average quoted spread. For each value of $\sigma$, average quoted spread $\overline{QS}$ and the average quoted spread in the Glosten-Milgrom Benchmark (dashed line in 5) is plotted.

Consider the adverse selection case first for a Q-learning AM in Panel A of figure 5. For all values of $\sigma$, the expected quoted spread in this case remains largely above the Glosten-Milgrom Benchmark. Notably, this is also the case when there is no adverse selection. These observations confirm for a broader set of parameters that AMs using Q-learning algorithms settle on non-competitive prices, failing to learn that they can increase their average profit by undercutting their competitor. Additionally, it is observed that the expected quoted spread increases with $\sigma$, the dispersion of clients' liquidity shock. This pattern is completely different from the Glosten-Milgrom Benchmark in which the quoted spread either decreases with $\sigma$ (adverse selection) or is zero for all values of $\sigma$ (no adverse selection). This finding is in line with the fact that, Q-learning is prone to the "Over-Pruning Bias" described earlier in this Section. As noise trading shocks increase, $\sigma$ in this case, aggressive trading strategies are prematurely pruned. For this reason, the AMs settle on higher spreads as $\sigma$ increases.

For a DDPG AM in Panel B of figure 5, the results are different from the Q-learning AM. In the adverse selection case, for all values of $\sigma$, the algorithm closely approximates the benchmark shown as the blue dotted line. The result is similar in the no adverse selection case. The algorithm closely approximates the benchmark yellow dotted line (at 0). This result shows that AMs using DDPG algorithm are considerably more effective at settling on more competitive prices and avoid the collusion problem observed among AMs using Q-learning algorithms.

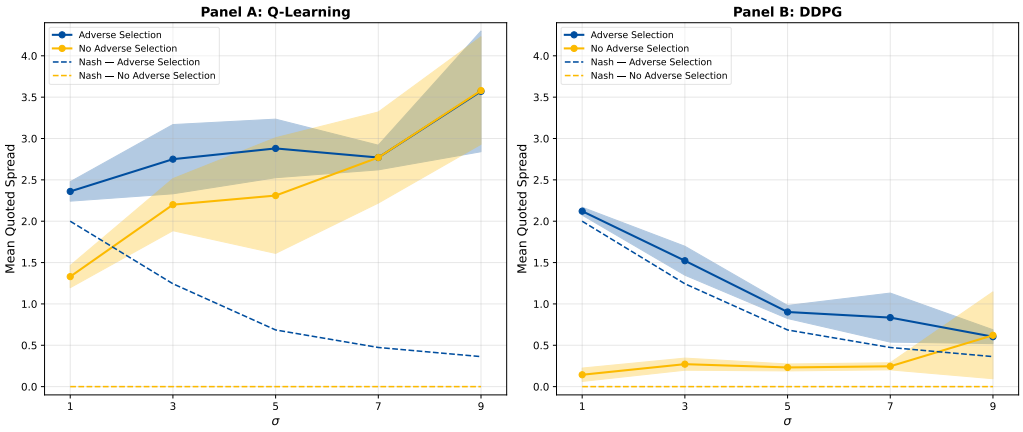


**Figure 5. Average Quoted Spread $\overline{QS}$, for different values of the dispersion of clients' liquidity shocks $\sigma$.** Other Parameters: $\Delta v = 4, N = 2, \mu = \frac{1}{2}, \mathbb{E}(v) = 2, T_Q = 500,000, T_D = 154,000, \tilde{T} = 10,000, K = 10$. This figure plots the Average Quoted Spread $\overline{QS}$ with 95% confidence intervals for Q-learning (Panel A) and DDPG (Panel B) agents in the adverse selection case and the no adverse selection case over 10 experiments, for different values of the dispersion of clients' liquidity shock $\sigma$. Glosten-Milgrom benchmark of Section 3.3 for both adverse selection and no adverse selection cases are shown as dotted lines.

Turning now to the expected realized spread, figure 6 shows for a Q-learning AM (Panel A) and a DDPG AM (Panel B) the effect of the dispersion of clients' liquidity shock $\sigma$ on the AMs' average realized spread.

Consider the adverse selection case first for a Q-learning AM in Panel A of Figure 6. For all values of $\sigma$, the expected realized spread remains substantially above the Glosten-Milgrom benchmark, mirroring the pattern observed for the quoted spread. This is also the case when there is no adverse selection. These observations further confirm that AMs using Q-learning algorithms consistently settle on non-competitive prices across a broader set of parameters. In contrast to the Glosten-Milgrom benchmark, where the realized spread decreases with $\sigma$ under adverse selection and is zero under no adverse selection, the Q-learning AM produces realized spreads that increase with $\sigma$, suggesting the algorithms collude without any direct communication.

For a DDPG AM in Panel B of Figure 6, the results are noticeably different. In the adverse selection case, the realized spread closely tracks the Glosten-Milgrom benchmark across all values of $\sigma$, consistent with the findings for the quoted spread. Under no adverse selection, the algorithm simi-

larly approximates the benchmark of zero. These results reinforce the conclusion from the previous sections that AMs using the DDPG algorithm are considerably more effective at learning competitive pricing strategies than their Q-learning counterparts, producing realized spreads that are broadly consistent with the theoretical predictions of the Glosten-Milgrom framework.

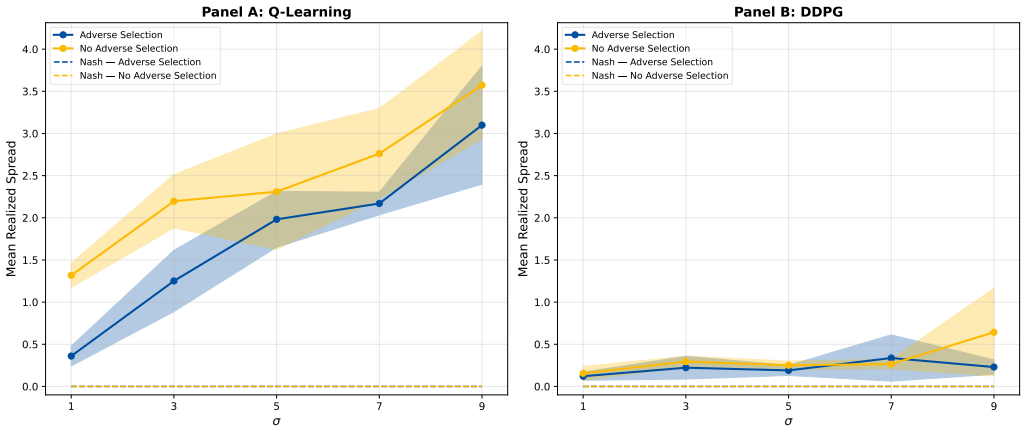


**Figure 6. Average Realized Spread $\overline{RS}$, for different values of the dispersion of clients' liquidity shocks $\sigma$.** Other Parameters: $\Delta v = 4, N = 2, \mu = \frac{1}{2}, \mathbb{E}(v) = 2, T_Q = 500,000, T_D = 154,000, \tilde{T} = 10,000, K = 10.$ This figure plots the Average Realized Spread $\overline{RS}$ with 95% confidence intervals for Q-learning(Panel A) and DDPG (Panel B) agents in the adverse selection case and no adverse selection case over 10 experiments, for different values of the dispersion of clients' liquidity shock $\sigma$. Glosten-Milgrom benchmark of Section 3.3 for both adverse selection and no adverse selection cases are shown as dotted lines.

## 8.2 Additional Algorithmic Market Makers

In this section I examine the impact of increasing the number of competing AMs. In particular, I investigate whether a large number of AMs induce more competitive pricing behaviour and drive expected spreads closer to the Glosten-Milgrom Benchmark and whether this effect differs between Q-learning and DDPG algorithms. Brogaard and Garriott (2019) empirically find that increase in the competition improves liquidity on stocks. They argue that the average bid-ask spread gradually declines with new entries of high-frequency traders in the market. Calvano et al. (2020) also predict that collusion is harder to sustain when the market is more fragmented. In fact, as the number of AMs increase, the expected gains from undercutting becomes larger ceteris paribus because the AMs' profits tied to the same price is divided among dealers. Additionally, the variance of these profits also decreases for the same reason.

To test this conjecture, I run experiments varying the number of AMs from 2 to 5, $N = \{2, 3, 4, 5\}$, for both algorithms to see which AM performs better under this assumption and report the results in figure 7 and figure 8.

Figure 7 plots the mean quoted spread $\overline{QS}$ as a function of the number of competing $AMs$ $N \in \{2,3,4,5\}$, under adverse selection and no adverse selection cases. Consider Panel A for the Q-learning algorithm, AM's average quoted spread declines gradually with increase in number of AMs reflecting the above claim by Brogaard and Garriott (2019). This is the same phenomenon observed in both adverse and no adverse selection. However, for all values $N$, the mean quoted spread remains substantially above the Glosten-Milgrom Benchmark shown as the dotted line in both adverse and no adverse selection cases. Even at $N = 5$, the Q-learning algorithm fails to converge at the equilibrium quoted spread, suggesting that increase in market competition alone is insufficient to eliminate supra-competitive pricing behaviour documented in the $N = 2$ baseline case. The standard deviation band is notably wide, reflecting high variance in convergence outcomes of the Q-learning algorithm.

Panel B presents a different picture for the DDPG AM. In the adverse selection case, the mean quoted spread tracks the Glosten-Milgrom Benchmark closely across all values of $N$. The absence of a pronounced decline with increase in $N$ suggests that the DDPG algorithm has largely learned the competitive pricing strategy in the $N = 2$ case, any additional competition provides only a marginal improvement. In the no adverse selection case, the algorithm similarly approximates the benchmark of zero across all values of $N$. The confidence interval band is considerably narrower than in Panel A, showing greater consistency of DDPG across experiments and its robustness to the over-pruning bias that affects Q-learning.

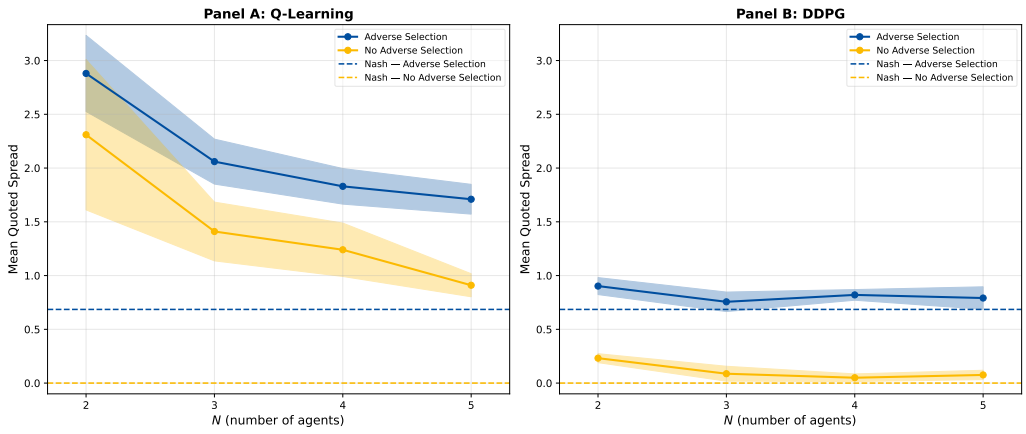


**Figure 7. Average Quoted Spread $\overline{QS}$, for different values of the number $N$ of AMs.** Other Parameters: $\sigma = 5, \Delta v = 4, \mu = \frac{1}{2}, \mathbb{E}(v) = 2, T_Q = 500,000, T_D = 154,000, \tilde{T} = 10,000, K = 10$. This graph plots the Average Quoted Spread with 95% confidence intervals for Q-learning (Panel A) and DDPG (Panel B) agents in the adverse selection case and no adverse selection case over 10 experiments, for different values of the number of $N$ of $AMs$. Glosten-Milgrom benchmark of Section 3.3 for both adverse selection and no adverse selection cases are shown as dotted lines.

Turning now to realized spread, Figure 8 presents the mean realized spread $\overline{RS}$ as a function of number of competing $AMs$ $N \in \{2, 3, 4, 5\}$ for both algorithms, under adverse selection and no adverse selection. Panel A for the Q-learning algorithm shows the mean quoted spread declines monotonically as $N$ increases in both adverse and no adverse selection cases, mirroring the pattern observed in Figure 7. Furthermore, for all values of $N$, the realized spread remains substantially above the GM Benchmark of zero. The confidence interval band is wide at $N = 2$ and narrows as $N$ increases, consistent with the assumption that pricing tightens as competition intensifies. This further confirms that the Q-learning algorithm is not learning the competitive pricing strategy implied by the Nash equilibrium, but rather converging to a supra-competitive outcome driven by the learning bias.

Panel B presents a different picture for the DDPG algorithm. The mean realized spread is close to zero across all values of $N$ in both adverse selection and no adverse selection cases, closely tracking the GM benchmark of zero throughout. At $N = 2$, the realized spread is marginally above zero, suggesting some degree of implicit collusion in the two-dealer case. However, this collusion dissipates rapidly as $N$ increases, with the algorithm converging to near-zero realized spreads from $N = 3$ onwards. The confidence interval band is considerably narrower than in Panel A across all values of $N$, reflecting greater consistency of the DDPG algorithm across experiments.

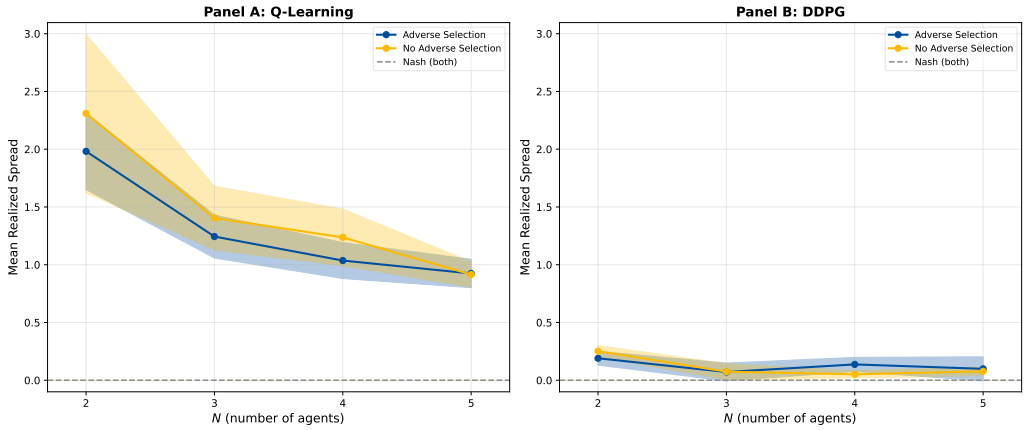


**Figure 8. Average Realized Spread $\overline{RS}$, for different values of the number $N$ of AMs.** Other Parameters: $\sigma = 5, \Delta v = 4, \mu = \frac{1}{2}, \mathbb{E}(v) = 2, T_Q = 500,000, T_D = 154,000, \tilde{T} = 10,000, K = 10$. This graph plots the Average Realized Spread with 95% confidence intervals for Q-learning (Panel A) and DDPG (Panel B) agents in the adverse selection case and no adverse selection case over 10 experiments, for different values of the number of $N$ of $AMs$. Glosten-Milgrom benchmark of Section 3.3 for both adverse selection and no adverse selection cases are shown as dotted lines.

These results reinforce the conclusion that the DDPG algorithm is considerably more effective at learning competitive pricing strategies than Q-learning across a range of market structures, and

that the implicit collusion observed in Q-learning is driven by a systematic learning bias rather than the number of market participants alone.

Taken together, Sections 8.1 and 8.2 present a consistent picture. The Q-learning algorithm persistently produces supra-competitive quoted and realized spreads across all market structures examined, with increasing competition only providing partial relief from the collusion driven by the "Over-Pruning Bias" and increasing liquidity shock of the client just amplifying the phenomenon. The DDPG algorithm, by contrast, closely approximates the Glosten-Milgrom Benchmark in both spread measures across all market structures, demonstrating that its continuous action spaces and actor-critic architecture are sufficient to overcome the collusive bias that affects Q-learning.

# 9 Discussion

This thesis provides results that demonstrate that the choice of reinforcement learning algorithm has first-order consequences for market outcomes in a version of the market making game (Glosten and Milgrom 1985) given by Colliard et al. (2026). Two dealers playing an identical game under identical market conditions converge to fundamentally different strategies depending on the algorithm they employ. Q-learning market makers settle on supra-competitive prices consistent with implicit collusion, while DDPG market makers converge close to the Glosten-Milgrom Benchmark price. This finding extends the implicit collusion of Q-learning agents observed by Calvano et al. (2020) and Dou et al. (2025) by showing that algorithmic collusion is not an inherent property of reinforcement learning per se, but rather a consequence of the specific learning architecture employed. The overfitting bias due to "Artificial Stupidity" observed by Dou et al. (2025) provides a compelling mechanism for why Q-learning is susceptible to implicit collusion in the model seen in this thesis and the results suggest that continuous action space and neural function approximation of DDPG are sufficient to overcome this bias.

## 9.1 Limitations and Scope for further Research

The findings of this thesis include several limitations which need to be studied. The version of the market making game (Glosten and Milgrom 1985; Colliard et al. 2026) employs a binary asset value $v_t \in \{v_L, v_H\}$ which is a strong simplification of the continuous asset value distributions observed in real financial markets.

The finding that DDPG does not produce implicitly collusive outcomes does not imply that all neural network based reinforcement learning algorithms are "collusion-free". The present thesis is

only limited to one simple model which compares only Q-learning and DDPG — many other algorithms exist, including but not limited to Proximal Policy Optimisation (PPO), Twin Delayed Deep Deterministic Policy Gradient (TD3), and Soft Actor-Critic (SAC), each with different properties that may affect their ability to tackle the collusion problem. The extent to which the present findings generalise across a broader class of reinforcement learning algorithms remains to be tested.

Validation of the simulation results against field data would be a valuable contribution; identifying whether real market makers using different algorithmic architectures produce different spreads would provide direct evidence for or against the theoretical results documented in this thesis, and would strengthen the policy implications of the present findings.

Colliard et al. (2026) suggest extending the model in such a way in which both dealers and clients use such algorithms, e.g., with clients using algorithms to choose the timing of their submission strategy, or whether to post limit or market orders. In the experiments presented in this thesis and Colliard et al. (2026), quotes are posted only by algorithms, whereas in actual security markets algorithms coexist with humans. An interesting avenue for further research is therefore to run experiments in which prices are set both by reinforcement learning algorithms and humans.

Finally, there is an important computational consideration on the practical applicability of the two algorithms in more realistic settings. In the current low-dimensional framework, Q-learning is computationally efficient — the discrete Q-matrix is small and updates are inexpensive. However, as state and action spaces grow in complexity, the Q-matrix grows exponentially, leaving tabular Q-learning less viable in high-dimensional environments. DDPG (Lillicrap et al. 2019), by contrast, is considerably more computationally demanding even in the simple setting of the present thesis — as evidenced by the substantially longer simulation runtimes relative to Q-learning but its neural function approximation scales more naturally to complex, high-dimensional state and actions spaces. This uncovers the possibility that DDPG may perform even better relative to Q-learning in more realistic market environments where the state space includes more complex variables. Whether the competitive advantage of DDPG documented in this thesis persists and potentially strengthens in such environments remains an open question. Nevertheless, the findings of the thesis suggest that the additional computational cost of DDPG is justified even in the simple setting considered here and if the algorithm's superiority proves robust to greater environment complexity, the case for employing such computational resources in real market making applications would be considerably strengthened.

# 10 Conclusion

This thesis conducts experiments in which Q-learning and Deep Deterministic Policy Gradient algorithms act as market makers in a setting given by Colliard et al. (2026) similar to Glosten and Milgrom (1985). I find that despite the challenges posed by an environment with adverse selection, the algorithmic market makers (AMs) exhibit realistic behaviour: their quoted spreads reflect adverse selection costs. However, it is observed that due to the simplicity of the Q-learning algorithm, AMs do not learn to undercut each other down to competitive price levels. Dou et al. (2025) argue that this failure arises from "Artificial Stupidity" among Q-learning agents which leads them to collude without any explicit communication. Moreover, AMs using a more complex algorithm like Deep Deterministic Policy Gradient (Lillicrap et al. 2019) learn to undercut each other and settle on more competitive price levels.

These findings hold up to change in underlying market parameters. Increasing the number of competing dealers $N$ drives Q-learning prices closer to the benchmark but does not fully eliminate the gap even at $N = 5$, while DDPG prices remain close to the Nash equilibrium regardless of market structure. Similarly, varying the dispersion of clients' liquidity shocks $\sigma$ does not alter the conclusion that Q-learning AMs consistently quote prices above the Glosten-Milgrom benchmark (Glosten and Milgrom 1985; Colliard et al. 2026) across all values of $\sigma$ as $\sigma$ increases, consistent with the signal-to-noise mechanism described by Colliard et al. (2026). The DDPG algorithm, by contrast, tracks the benchmark closely across all values of $\sigma$, suggesting its more complex architecture is more robust to noisy feedback than the simple architecture of Q-learning.

Several limitations should be noted when interpreting these results. The analysis is conducted within a market-making framework that, by design, abstracts from many features of real-world financial markets: dealers interact symmetrically over a binary-valued asset, the client population is static and neither inventory nor multi-period strategic interaction is modelled. Additionally, the Q-learning algorithm operates over a discretised price grid while DDPG acts on a continuous action space, so part of the observed performance difference may reflect differences in the underlying action-space structure rather than purely algorithm complexity. Extending the model to comparisions among algorithms utilising same action-space structures and involving richer state spaces such as order flow imbalance, inventory positions, or dealers' beliefs about one another's algorithms would constitute a more fair of test of each algorithm's capabilities and is a natural avenue for future work.

Overall, the results of this thesis suggest that in a world where prices are increasingly set by algorithms (SEC 2020), the design of those algorithms matters and not just for the firms that deploy them, but for the quality and efficiency of the markets in which they operate. Whether this result gen-

eralises to more sophisticated market environments and a broader class of reinforcement learning algorithms remains an important open question for further research.

From a policy perspective, these findings carry a cautious message. The result that Q-learning dealers arrive at supra-competitive prices through sheer algorithmic simplicity without any communication or explicit coordination raises the concern that collusive outcomes may be difficult to detect or prosecute under existing antitrust frameworks, since no intent or coordination is required (Dou et al. 2025). At the same time, the DDPG result offers a more optimistic view: markets populated by more sophisticated algorithms can, in principle, self-organise toward competitive outcomes. An important open question is whether this holds in environments with *heterogeneous* agents, where Q-learning and DDPG dealers interact in the same market, and whether the competitive discipline of the more sophisticated agent is strong enough to undermine the tacit collusion of the simpler one. Answering this question would have direct implications for how regulators think about algorithm auditing and the design of market surveillance tools for increasingly AI-driven markets.

# References


Glosten, Lawrence R., and Paul R. Milgrom. 1985. "Bid, Ask and Transaction Prices in a Specialist Market with Heterogeneously Informed Traders." *Journal of Financial Economics* 14 (1): 71–100. [1, 3, 5, 7, 8, 32, 34]

Kyle, Albert S. 1985. "Continuous auctions and insider trading." *Econometrica: Journal of the Econometric Society,* 1315–1335. [4]

Watkins, Christopher JCH, and Peter Dayan. 1992. "Q-learning." *Machine learning* 8 (3): 279–292. [2, 10]

Sutton, Richard S, Andrew G Barto, et al. 1998. *Reinforcement learning: An introduction.* Vol. 1. 1. MIT press Cambridge. [1, 10, 11]

Huck, Steffen, Hans-Theo Normann, and Jörg Oechssler. 2004. "Two are few and four are many: number effects in experimental oligopolies." *Journal of economic behavior & organization* 53 (4): 435–446. [3]

Hasbrouck, Joel, and Gideon Saar. 2013. "Low-latency trading." High-Frequency Trading, *Journal of Financial Markets* 16 (4): 646–679. https://doi.org/https://doi.org/10.1016/j.finmar.2013.05.003. [12]

Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. 2013. "Playing atari with deep reinforcement learning." *arXiv preprint arXiv:1312.5602.* [13]

Kingma, Diederik P, and Jimmy Ba. 2014. "Adam: A method for stochastic optimization." *arXiv preprint arXiv:1412.6980.* [18, 20]

Silver, David, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. 2014. "Deterministic policy gradient algorithms." In *International conference on machine learning,* 387–395. Pmlr. [14]

Brogaard, Jonathan, and Corey Garriott. 2019. "High-Frequency Trading Competition." *Journal of Financial and Quantitative Analysis* 54 (4): 1469–1497. https://doi.org/10.1017/S0022109018001175. [29, 30]

Li, Yang, Wanshan Zheng, and Zibin Zheng. 2019. "Deep robust reinforcement learning for practical algorithmic trading." *IEEE Access* 7:108014–108022. [4]

Lillicrap, Timothy P., Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2019. *Continuous control with deep reinforcement learning.* arXiv: 1509.02971 [cs.LG]. https://arxiv.org/abs/1509.02971. [2–4, 10, 12, 13, 18, 20, 33, 34]

Zhang, Zihao, Stefan Zohren, and Stephen Roberts. 2019. "Deep reinforcement learning for trading." *arXiv preprint arXiv:1911.10107.* [4]

Calvano, Emilio, Giacomo Calzolari, Vincenzo Denicolo, and Sergio Pastorello. 2020. "Artificial intelligence, algorithmic pricing, and collusion." *American Economic Review* 110 (10): 3267–3297. [2, 4, 5, 16, 20, 25, 29, 32]

Gu, Shihao, Bryan Kelly, and Dacheng Xiu. 2020. "Empirical asset pricing via machine learning." *The Review of Financial Studies* 33 (5): 2223–2273. [4]

SEC. 2020. "Staff Report on Algorithmic Trading in U.S. Capital Markets." Technical report. U.S. Securities and Exchange Commission. Accessed January 1, 2025. https://www.sec.gov/files/algo_trading_report_2020.pdf. [1, 34]

Goldstein, Itay, Chester S Spatt, and Mao Ye. 2021. "Big data in finance." *The Review of Financial Studies* 34 (7): 3213–3225. [1]

Kelly, Bryan, and Dacheng Xiu. 2023. "Financial machine learning." *Foundations and Trends® in Finance* 13 (3-4): 205–363. [4]

Abada, Ibrahim, Xavier Lambin, and Nikolay Tchakarov. 2024. “Collusion by mistake: Does algorithmic sophistication drive supra-competitive profits?” *European Journal of Operational Research* 318 (3): 927–953. [4]

Asker, John, Chaim Fershtman, and Ariel Pakes. 2024. “The impact of artificial intelligence design on pricing.” *Journal of Economics & Management Strategy* 33 (2): 276–304. [4, 16]

Dou, Winston Wei, Itay Goldstein, and Yan Ji. 2025. “AI-Powered Trading, Algorithmic Collusion, and Price Efficiency.” Working Paper, Working Paper Series 34054. National Bureau of Economic Research, July. https://doi.org/10.3386/w34054. [2–5, 25, 26, 32, 34, 35]

Gufler, Ivan, Francesco Sangiorgi, and Emanuele Tarantino. 2025. “(Deep) Learning to Trade: An Experimental Analysis of AI Trading and Market Outcomes.” *Available at SSRN 5375160*. [13]

Colliard, Jean-Edouard, Thierry Foucault, and Stefano Lovo. 2026. “Algorithmic Pricing and Liquidity in Securities Markets.” *The Review of Financial Studies* (February): hhag010. https://doi.org/10.1093/rfs/hhag010. eprint: https://academic.oup.com/rfs/advance-article-pdf/doi/10.1093/rfs/hhag010/66966781/hhag010.pdf. [1–9, 11, 12, 15–17, 20, 25, 26, 32–34]

European Securities and Markets Authority (ESMA). 2026. “Supervisory Briefing on Algorithmic Trading in the EU.” Technical report. Paris, France: ESMA. https://europa.eu. [16]

# Appendix

## A.1 Training Dynamics of Agent 2

This appendix presents the training dynamics of $AM_2$ for completeness. As noted in Section 7.1, although $AM_1$ and $AM_2$ follow different training trajectories due to independent random initialisation and stochastic exploration paths, both agents converge to virtually identical exploitation phase prices in all experiments, consistent with the symmetric structure of the market making game. The figures presented here are qualitatively identical to those of $AM_1$ reported in Section 7.1.

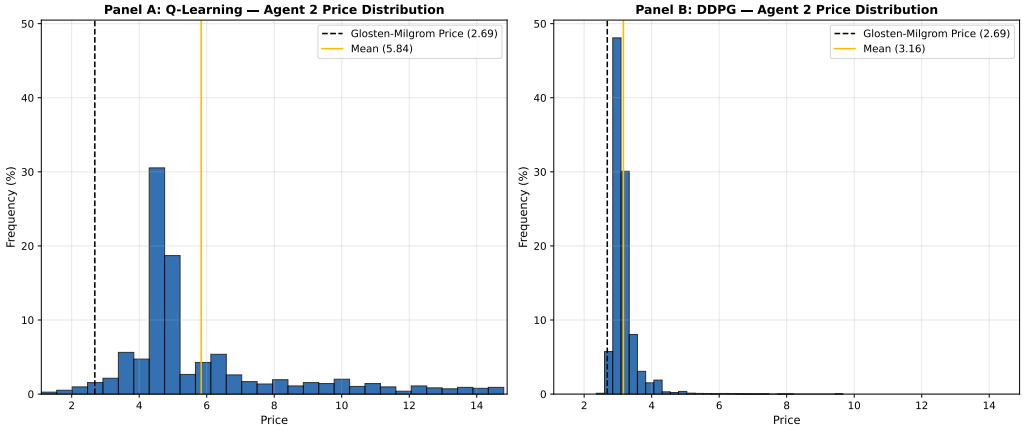


**Figure A.1. Distribution of the greedy price of $AM_2$ in adverse selection case.** Parameters: $\sigma = 5, \Delta v = 4, N = 2, \mu = \frac{1}{2}, \mathbb{E}(v) = 2, T_Q = 500,000, T_D = 154,000, \tilde{T} = 10,000, K = 10$. This figure shows a histogram of the greedy price of $AM_2$ for Q-learning (left) and DDPG (right). For each possible price $a$ between 1.1 and 14.9 the bar indicates the percentage of 500,000 iterations for Q-learning and 154,000 iterations for DDPG conducted in which $a_2^* = a$.

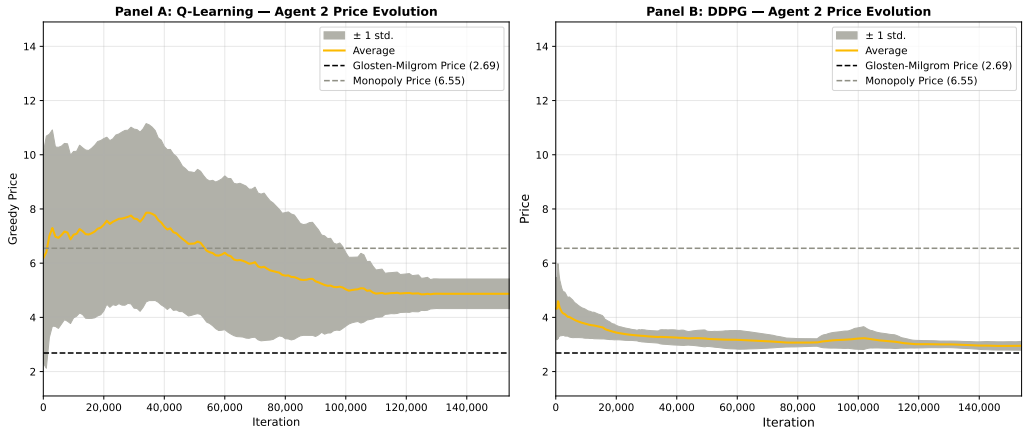


**Figure A.2. Price evolution of greedy price of $AM_2$ in adverse selection case.** Parameters: $\sigma = 5$, $\Delta v = 4$, $N = 2$, $\mu = \frac{1}{2}$, $\mathbb{E}(v) = 2$, $T_Q = 500,000$, $T_D = 154,000$, $\tilde{T} = 10,000$, $K = 10$. The left panel of this figure shows the price evolution of Q-learning agent 2 for each iteration $t$ between 1 and 500,000 the average of $AM_2$'s greedy price $a^*_{2,t}$. The right panel of this figure shows the price evolution of DDPG agent 2 for each iteration $t$ between 1 and 154,000 the average greedy price $a^*_{2,t}$ of $AM_2$. The solid line represents a 10% moving average for better readability. As a measure of dispersion, standard deviation of $a^*_2$ across experiments is computed and the average of $a^*_2$ plus/minus one standard deviation is plotted for both panels.

## A.2 DDPG Training Loss Functions

Figure A.3 presents the actor and critic loss functions of the DDPG algorithm averaged across $K = 10$ independent experiments, smoothed using a 1,000 iteration moving average. These diagnostics serve as evidence that the DDPG algorithm has converged to a stable pricing policy by the end of the training phase.

Panel A shows the actor loss, defined as the negative mean Q-value of the actor's chosen actions:

$$\mathcal{L}_{\text{actor}} = -\frac{1}{|B|}\sum_{i \in B} Q(s_i, \mu(s_i|\theta^{\varphi})|\theta^{Q}) \tag{21}$$

The actor loss declines sharply from an initial value of approximately 0.7 to a stable plateau of around $-0.10$ to $-0.20$ within the first 60,000 iterations, where it remains for the remainder of training. Unlike a supervised learning loss which converges to zero, the actor loss converges to a negative plateau reflecting the stable Q-value of the learned pricing strategy when the actor is no longer finding significantly more profitable pricing actions. The gradual upward drift toward $-0.15$ in the first half of training reflects fine-tuning of the policy as the exploration noise gradually decays and approaches its minimum value of $\sigma_{\text{final}} = 0.01$ in the latter half of training, rather than a deterioration of the learned strategy.

Panel B shows the critic loss, defined as the mean squared error between the predicted Q-value and the target value:

$$\mathcal{L}_{\text{critic}} = \frac{1}{|B|} \sum_{i \in B} \left( Q(s_i, a_i | \theta^Q) - y_i \right)^2 \tag{22}$$

The critic loss starts from approximately $0.7$ and drops to near zero and then rises from near zero to a plateau of approximately 1.0 by iteration 20,000, after which it remains broadly stable for the remainder of training. The initial fall and rise reflects the critic accumulating sufficient transitions in the replay buffer $\mathcal{D}$ to form meaningful Q-value estimates. Prior to the buffer filling, the critic has limited data to learn from and its loss is artificially low. The subsequent stabilisation indicates the critic has converged to a consistent approximation of the expected profit function.

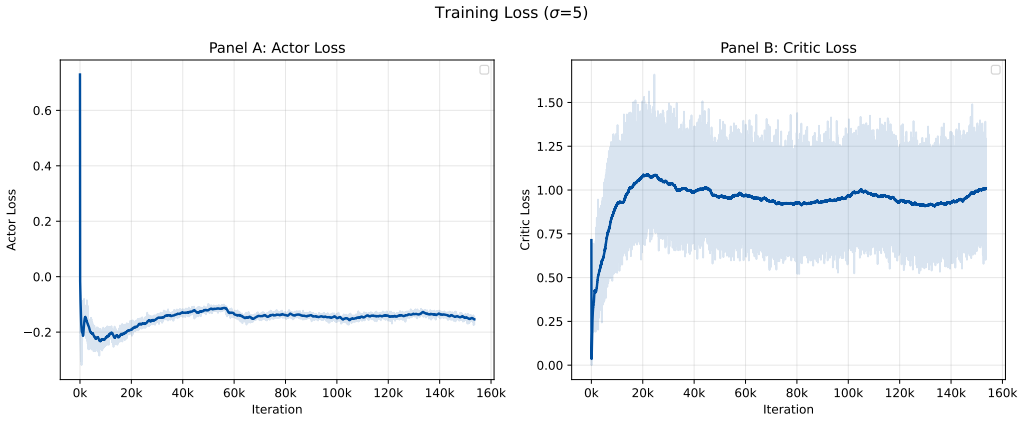


**Figure A.3. Actor and Critic Loss functions:** Panel A of the figure shows the Actor loss over 154,000 iterations for a DDPG AM. Panel B of the figure plots the critic loss for a DDPG AM over 154,000 iterations. Both panels include a 1,000 iteration moving average solid blue line for better readability with the raw signal in light blue.

Together, the stabilisation of both loss functions confirms that the DDPG algorithm converges to a stable pricing policy well within the allocated training budget of $T_D = 154,000$ iterations, justifying the use of exploitation phase prices as the primary outcome measure in Section 7.2.

## A.3 Code and Reproducibility

The full code used to produce the results reported in this thesis is publicly available on GitHub. This includes the implementation of the Q-learning and DDPG algorithms, the market making environment, all simulation scripts, and the plotting functions used to generate the figures throughout the thesis.

https://github.com/AdiGosw/msc_thesis

## Declaration

I hereby confirm that the work presented has been performed and interpreted solely by myself except for where I explicitly identified the contrary. I assure that this work has not been presented in any other form for the fulfillment of any other degree or qualification. Ideas taken from other works in letter and in spirit are identified in every single case.

June 5, 2026

Aditya Goswami